BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC CẦN THƠ

TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

BÁO CÁO NGHIÊN CỨU KHOA HỌC SINH VIÊN

XÂY DỰNG ỨNG DỤNG QUẢN LÝ CÔNG VIỆC THEO KIẾN TRÚC ĐA THUÊ BAO

Mã số đề tài THS2026-67

Chủ nhiệm đề tài Ngô Hưng Thịnh

Giảng viên hướng dẫn TS. Ngô Bá Hùng

Sinh viên thực hiện

Ngô Hưng Thịnh

Nguyễn Thanh Trọng

Trang Minh Chánh

Huỳnh Hồng Ân

Cao Tường Hưng

Cần Thơ tháng 11 năm 2026

Tóm tắt

Đề tài nghiên cứu kiến trúc đa thuê bao thông qua bài toán quản lý công việc Kanban trong môi trường đại học. Báo cáo hệ thống hóa khái niệm, so sánh các cách chia sẻ và cô lập tài nguyên, sau đó đề xuất ứng dụng có khả năng cấu hình và phục vụ nhiều tổ chức trên mã nguồn, backend và cơ sở dữ liệu dùng chung. Tenant ID, ngữ cảnh đã xác minh và metadata lúc chạy được kết hợp để duy trì dữ liệu, quyền và cấu hình riêng theo workspace. Phần thiết kế cụ thể hóa các cơ chế thành mô-đun, lược đồ dữ liệu và luồng nghiệp vụ; ma trận kiểm chứng xác định bằng chứng cần thu thập khi hiện thực. Khả năng mở rộng ngang được đặt ở hướng phát triển.

Từ khóa: multi-tenancy; SaaS; Kanban; Tenant ID; TenantContext; metadata runtime.

Mở đầu

Giảng viên và sinh viên cần một công cụ tổ chức, giao việc và theo dõi tiến độ cho nhiều nhóm học tập hoặc nghiên cứu. Triển khai một nền tảng dùng chung giúp tập trung vận hành, nhưng đặt ra yêu cầu giữ riêng thành viên, dữ liệu và cấu hình của từng nhóm. Đề tài lựa chọn workspace làm đơn vị tenant và Kanban làm nghiệp vụ minh họa cho các cơ chế kiến trúc này.

Mục tiêu là đề xuất thiết kế, xây dựng và kiểm chứng ứng dụng có khả năng cấu hình và phục vụ đa thuê bao hiệu quả. Câu hỏi nghiên cứu tập trung vào cách xác định đúng tenant trên từng yêu cầu, cô lập dữ liệu trên bảng chung và áp dụng biến thể nghiệp vụ qua metadata mà không tạo mã hoặc bản chạy riêng. Phạm vi gồm ứng dụng Web và triển khai thử nghiệm trên máy chủ ảo; không phát triển sản phẩm thương mại đầy đủ hoặc hạ tầng phân tán quy mô lớn.

Phương pháp kết hợp tổng hợp tài liệu, phân tích yêu cầu trong thuyết minh, thiết kế kiến trúc và xác định các tình huống kiểm chứng có thể tái lập. Báo cáo gồm ba chương: cơ sở lý thuyết; so sánh cơ chế và đánh đổi; phân tích, thiết kế ứng dụng. Các kết quả kỳ vọng trong ma trận kiểm chứng là điều kiện đánh giá; kết luận thực nghiệm cần được bổ sung từ quá trình chạy thử thực tế.

Mục lục

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT VÀ MÔ HÌNH HÓA KIẾN TRÚC ĐA THUÊ BAO (MULTI-TENANCY) 7
1.1. Tổng quan về SaaS và tiến trình tiến hóa của các mô hình chia sẻ tài nguyên 7
1.1.1. Khái niệm và đặc điểm của mô hình Phần mềm dưới dạng Dịch vụ (Software as a Service – SaaS) 7
1.1.2. Tiến trình tiến hóa từ chia sẻ thời gian, ASP và ứng dụng Web đa người dùng đến SaaS đa thuê bao 7
1.1.3. Mối quan hệ kinh tế – kỹ thuật giữa SaaS và kiến trúc Multi-Tenancy 8
1.1.4. Phân tích Chi phí sở hữu (TCO) từ góc nhìn nhà cung cấp dịch vụ 8
1.2. Các khái niệm và thuật ngữ nền tảng trong kiến trúc Multi-Tenancy 9
1.2.1. Khách hàng và Thuê bao (Customer & Tenant) 9
1.2.2. Người dùng và Thành viên thuê bao (User & Membership) 10
1.2.3. Dịch vụ và Ứng dụng (Service & Application) 10
1.2.4. Instance và các ví dụ về Web Server, Application Server, Database Server 10
1.2.5. Tài nguyên hệ thống (System Resources) 10
1.2.6. Định danh thuê bao và Siêu dữ liệu (Tenant ID & Metadata) 10
1.2.7. Định nghĩa Multi-Tenancy và mối quan hệ giữa các khái niệm nền tảng 11
1.3. Phân biệt và mô hình hóa các cách tổ chức hệ thống 11
1.3.1. Mô hình Đơn thuê bao (Single-Tenant – ST) 11
1.3.2. Mô hình Đa instance (Multi-Instance – MI) 11
1.3.3. Mô hình Đa người dùng (Multi-User – MU) 11
1.3.4. Mô hình Đa thuê bao (Multi-Tenant – MT) 12
1.3.5. Bảng đối chiếu đa tiêu chí và mô hình hóa trực quan ST, MI, MU và MT 12
1.4. Các cấp độ trưởng thành kiến trúc SaaS theo mô hình Chong & Carraro 13
1.4.1. Cấp độ 1 – Ad Hoc / Custom 13
1.4.2. Cấp độ 2 – Configurable 13
1.4.3. Cấp độ 3 – Configurable, Multi-Tenant-Efficient 13
1.4.4. Cấp độ 4 – Scalable, Configurable, Multi-Tenant-Efficient 14
1.4.5. Phân biệt Cấp độ 2 và Cấp độ 3 về cấu hình, instance và cơ sở dữ liệu 14
1.4.6. Bảng so sánh bốn cấp độ và định vị kiến trúc mục tiêu của đề tài 14
1.5. Mô hình hóa kiến trúc Multi-Tenancy theo ba tầng và ngăn xếp chín tầng 16
1.5.1. Tầng người thuê – Tổ chức thuê bao và người dùng 16
1.5.2. Tầng nhà cung cấp dịch vụ – Ứng dụng, nghiệp vụ và cấu hình 16
1.5.3. Tầng các instance và hạ tầng – Máy chủ ứng dụng và cơ sở dữ liệu 16
1.5.4. Quan hệ chia sẻ và tách biệt tài nguyên qua bốn cấp độ trưởng thành 17
1.5.5. Mô hình hóa tổng thể kiến trúc đa thuê bao và các cơ chế kiểm soát xuyên tầng 17
1.5.6. Ánh xạ mô hình tổng thể vào kiến trúc mục tiêu Cấp độ 3 18

1.5.7. Ngăn xếp chín tầng nguyên bản của Kabbedijk và cộng sự 18
1.5.8. Ngăn xếp chín tầng cập nhật cho kiến trúc Cloud-Native 19
1.6. Các đặc trưng kiến trúc cốt lõi của hệ thống Multi-Tenancy 21
1.6.1. Chia sẻ tài nguyên trong suốt (Transparent Resource Sharing)..... 21
1.6.2. Tùy biến động lúc chạy dựa trên siêu dữ liệu (Metadata-Driven Runtime Customization)..... 21
1.6.3. Cô lập dữ liệu, ngữ cảnh và tài nguyên giữa các thuê bao (Tenant Isolation)..... 21
1.6.4. Khả năng mở rộng, tự phục vụ và quản lý vòng đời thuê bao (Scalability, Self-Service & Tenant Lifecycle Management)..... 21
1.6.5. Đo lường và giám sát theo thuê bao (Tenant-Aware Metering & Monitoring)..... 21
1.6.6. Liên hệ các đặc trưng kiến trúc với tiêu chí kiểm chứng Cấp độ 3 22
1.7. Tổng kết chương..... 22
CHƯƠNG 2. KIẾN TRÚC ĐA THUÊ BAO — CƠ CHẾ CHIA SẺ, CÔ LẬP VÀ CÁC ĐÁNH ĐỔI 23
2.1. Nguyên lý tổ chức kiến trúc đa thuê bao 23
2.1.1. Chia sẻ và tách biệt tài nguyên 23
2.1.2. Ranh giới thuê bao và các phương diện cô lập 23
2.1.3. Quan hệ giữa mô hình triển khai và mức độ trưởng thành SaaS 23
2.2. Các mô hình triển khai kiến trúc đa thuê bao 23
2.2.1. Mô hình Silo 23
2.2.2. Mô hình Pool 24
2.2.3. Mô hình Bridge..... 25
2.2.4. Mô hình Hybrid và các biến thể mở rộng 25
2.2.5. So sánh đặc điểm và điều kiện áp dụng 26
2.3. Các cơ chế chia sẻ và cô lập 27
2.3.1. Tùy biến giao diện và nghiệp vụ bằng metadata 27
2.3.2. Tổ chức và cô lập dữ liệu..... 27
2.3.3. Quản lý ngữ cảnh và quyền truy cập theo tenant..... 27
2.3.4. Phân bổ tài nguyên và hạn chế ảnh hưởng giữa các tenant 28
2.3.5. Đánh đổi về chi phí, khả năng tùy biến và vận hành 28
2.4. Các kịch bản kiến trúc và lựa chọn phương án..... 28
2.4.1. Kịch bản ưu tiên chi phí và khởi tạo nhanh 28
2.4.2. Kịch bản ưu tiên cô lập và vận hành độc lập 29
2.4.3. Kịch bản phục vụ nhiều phân khúc thuê bao 29
2.4.4. So sánh đánh đổi và căn cứ lựa chọn..... 29
2.5. Kiến trúc có khả năng cấu hình và phục vụ đa thuê bao hiệu quả (Configurable, Multi-Tenant-Efficient) 29
2.5.1. Mô hình tổng thể và phạm vi áp dụng cho đề tài..... 29
2.5.2. Mã nguồn chung và instance ứng dụng dùng chung..... 30
2.5.3. Cơ sở dữ liệu dùng chung và phân tách bằng Tenant ID..... 30

2.5.4. Tùy biến runtime qua metadata và luồng xử lý theo tenant 30
2.6. Đánh giá và hướng phát triển kiến trúc 30
2.6.1. Tiêu chí và phương pháp kiểm chứng kiến trúc được lựa chọn 30
2.6.2. Giới hạn của phương án dùng chung 31
2.6.3. Định hướng mở rộng ngang và phân bố dữ liệu 31
2.7. Tổng kết chương 31
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ ỨNG DỤNG QUẢN LÝ CÔNG VIỆC ĐA THUÊ BAO 32
3.1. Phân tích bài toán và yêu cầu hệ thống 32
3.1.1. Bối cảnh sử dụng, các tác nhân và phạm vi ứng dụng 32
3.1.2. Yêu cầu chức năng và sơ đồ ca sử dụng 32
3.1.3. Yêu cầu phi chức năng 33
3.2. Thiết kế kiến trúc tổng thể 34
3.2.1. Kiến trúc có khả năng cấu hình và phục vụ đa thuê bao hiệu quả (Configurable, Multi-Tenant-Efficient) 34
3.2.2. Các thành phần hệ thống và lựa chọn công nghệ 34
3.2.3. Mã nguồn chung, instance dùng chung và luồng xử lý yêu cầu 35
3.3. Thiết kế cơ sở dữ liệu 36
3.3.1. Mô hình dữ liệu nghiệp vụ và quan hệ giữa các thực thể 36
3.3.2. Phân tách dữ liệu và kiểm soát truy cập theo Tenant ID 38
3.3.3. Lưu trữ cấu hình và metadata theo tenant 38
3.4. Thiết kế các cơ chế đa thuê bao 39
3.4.1. Khởi tạo, nhận diện và quản lý ngữ cảnh tenant 39
3.4.2. Phân quyền và cô lập dữ liệu giữa các tenant 39
3.4.3. Tùy biến giao diện và nghiệp vụ lúc chạy qua metadata 40
3.5. Thiết kế chức năng và giao diện ứng dụng 41
3.5.1. Quản lý không gian làm việc và thành viên 41
3.5.2. Quản lý dự án, bảng Kanban và trạng thái công việc 41
3.5.3. Chia sẻ tài nguyên, thông báo và giao diện theo vai trò 41
3.6. Thiết kế triển khai và kiểm chứng 42
3.6.1. Mô hình triển khai ứng dụng và cơ sở dữ liệu dùng chung 42
3.6.2. Đối chiếu yêu cầu thiết kế với tiêu chí kiểm chứng 43
3.7. Tổng kết chương 44
TÀI LIỆU THAM KHẢO 45

Danh mục hình

Hình 1.1 Các hướng phát triển của mô hình chia sẻ tài nguyên
Hình 1.2 Chi phí sở hữu của nhà cung cấp theo số tenant
Hình 1.3 Phân biệt đơn vị tổ chức của ST MI MU và MT
Hình 1.4 Tổ chức mã nguồn ứng dụng và dữ liệu qua bốn cấp độ trưởng thành
Hình 1.5 Quan hệ ba tầng trong cung ứng dịch vụ đa thuê bao
Hình 1.6 Mô hình tổng thể và các cơ chế kiểm soát xuyên tầng
Hình 1.7 Ngăn xếp chín tầng nguyên bản của Kabbedijk và cộng sự (vẽ lại theo [3])
Hình 1.8 Ngăn xếp chín tầng cập nhật cho kiến trúc Cloud-Native do nhóm đề xuất
Hình 2.1 Tài nguyên ứng dụng và dữ liệu dành riêng theo tenant
Hình 2.2 Ứng dụng và bảng dữ liệu dùng chung có nhận biết tenant
Hình 2.3 Chia sẻ ứng dụng và phân tách nguồn dữ liệu
Hình 2.4 Phối hợp tài nguyên dùng chung và dành riêng
Hình 3.1 Nhóm ca sử dụng theo vai trò của ứng dụng
Hình 3.2 Kiến trúc thành phần của ứng dụng được đề xuất
Hình 3.3 Luồng tạo thẻ có xác lập và giải phóng ngữ cảnh tenant
Hình 3.4 Lược đồ logic các thực thể dữ liệu cốt lõi
Hình 3.5 Cập nhật và áp dụng metadata theo tenant khi ứng dụng đang chạy
Hình 3.6 Thiết kế giao diện quản lý công việc theo workspace
Hình 3.7 Mô hình triển khai ứng dụng phục vụ nhiều tenant trên máy chủ thử nghiệm

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT VÀ MÔ HÌNH HÓA KIẾN TRÚC ĐA THUÊ BAO (MULTI-TENANCY)

Chương này làm rõ khái niệm, mô hình tổ chức và các cấp độ trưởng thành của kiến trúc đa thuê bao. Mục tiêu là xác định tài nguyên dùng chung, phạm vi riêng của tenant và những thuộc tính cần kiểm chứng, làm nền cho việc so sánh phương án ở Chương 2.

1.1. Tổng quan về SaaS và tiến trình tiến hóa của các mô hình chia sẻ tài nguyên

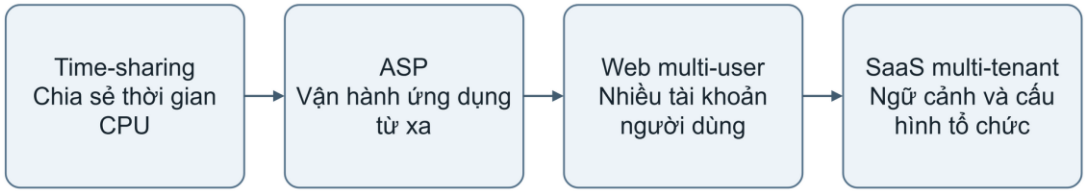
1.1.1. Khái niệm và đặc điểm của mô hình Phần mềm dưới dạng Dịch vụ (Software as a Service – SaaS)

Phần mềm dưới dạng dịch vụ (Software as a Service, SaaS) là cách cung cấp ứng dụng do nhà cung cấp triển khai và vận hành, còn khách hàng sử dụng chức năng qua mạng. NIST phân biệt SaaS với việc khách hàng trực tiếp quản lý hạ tầng nền; phạm vi cấu hình của khách hàng giới hạn ở những khả năng ứng dụng cho phép [1]. Nhà cung cấp chịu trách nhiệm duy trì dịch vụ, cập nhật và bảo trì trong suốt quá trình sử dụng.

SaaS mô tả phương thức cung ứng phần mềm, còn multi-tenancy mô tả cách nhiều thuê bao sử dụng chung tài nguyên. Vì vậy, một dịch vụ SaaS có thể cung cấp môi trường riêng cho từng khách hàng hoặc dùng chung ứng dụng giữa nhiều tổ chức. Hình thức thu phí định kỳ không tự quyết định kiến trúc triển khai.

1.1.2. Tiến trình tiến hóa từ chia sẻ thời gian, ASP và ứng dụng Web đa người dùng đến SaaS đa thuê bao

Các mô hình chia sẻ tài nguyên phát triển theo nhiều hướng có liên hệ. Time-sharing phân phối thời gian xử lý của máy tính cho nhiều tài khoản. ASP đưa ứng dụng lên môi trường do nhà cung cấp vận hành nhưng thường giữ các bản chạy riêng theo khách hàng. Ứng dụng Web đa người dùng mở rộng khả năng phục vụ nhiều cá nhân; SaaS đa thuê bao bổ sung phạm vi quản trị dữ liệu và cấu hình theo tổ chức [3].



Hình 1.1 Các hướng phát triển của mô hình chia sẻ tài nguyên

Các khối nêu đơn vị chia sẻ nổi bật ở mỗi hướng phát triển. Mũi tên là quan hệ kế thừa; điểm mới của SaaS đa thuê bao trong phạm vi báo cáo là quản lý dữ liệu và cấu hình ở cấp tổ chức.

Hình 1.1 biểu diễn sự kế thừa ý tưởng chia sẻ, không phải tiến trình thay thế tuyệt đối. On-premise, ASP và SaaS vẫn có thể cùng tồn tại. Khác biệt có ý nghĩa đối với đề tài là sự chuyển từ quản lý từng tài khoản hoặc bản cài đặt sang quản lý nhiều không gian làm việc độc lập trên một nền tảng chung.

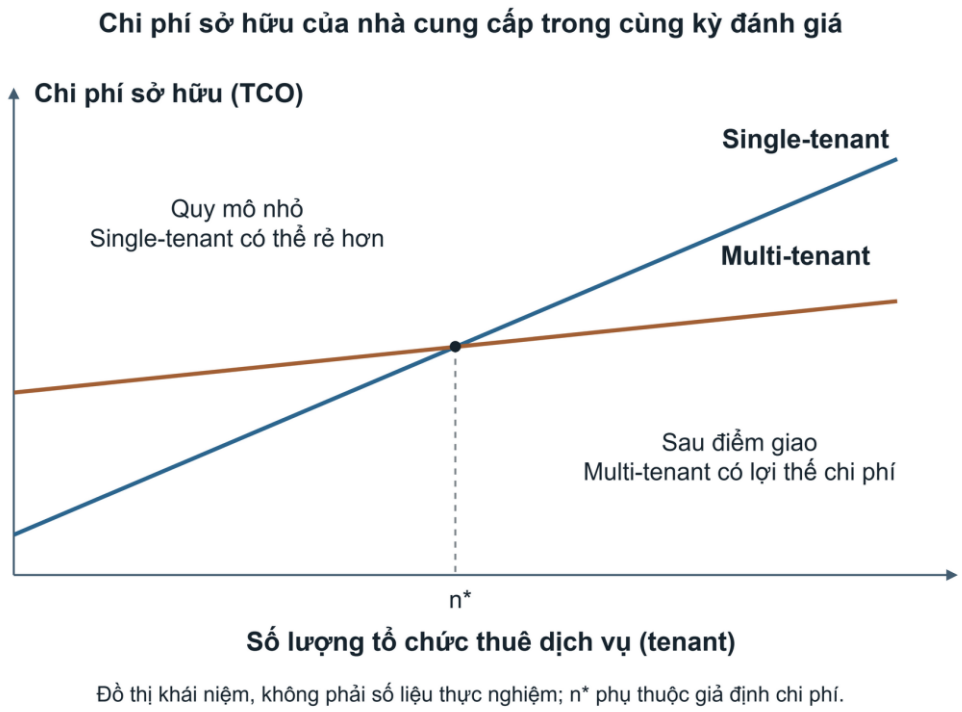
1.1.3. Mối quan hệ kinh tế – kỹ thuật giữa SaaS và kiến trúc Multi-Tenancy

Nhà cung cấp SaaS phải cân bằng chi phí phục vụ với nhu cầu riêng của từng tổ chức. Môi trường dành riêng thuận lợi cho quản lý cấu hình và đơn vị sao lưu, nhưng tăng số bản triển khai phải duy trì. Tài nguyên dùng chung tạo cơ hội tận dụng năng lực còn dư, giảm công việc cấp phát lặp lại và tập trung hóa cập nhật; đổi lại, phần mềm phải nhận biết tenant và thực thi cô lập nhất quán.

Hiệu quả chia sẻ phụ thuộc quy mô, tải và mức dịch vụ. Các tenant có đỉnh tải khác nhau có thể tận dụng chung năng lực xử lý; khi hệ thống đạt giới hạn, nhà cung cấp vẫn phải tăng tài nguyên. Vì vậy, giảm chi phí là động lực lựa chọn kiến trúc và cần được kiểm tra theo điều kiện vận hành, thay vì coi là kết quả mặc định.

1.1.4. Phân tích Chi phí sở hữu (TCO) từ góc nhìn nhà cung cấp dịch vụ

Chi phí sở hữu (Total Cost of Ownership, TCO) trong báo cáo được xét từ góc nhìn nhà cung cấp dịch vụ. Nó gồm chi phí xây dựng, triển khai và vận hành trong cùng một kỳ đánh giá: hạ tầng, bảo trì, giám sát, sao lưu và xử lý sự cố. Đây không phải giá thuê bao khách hàng trả. Trục hoành của Hình 1.2 là số tổ chức thuê dịch vụ; trục tung là tổng chi phí của nhà cung cấp.



Hình 1.2 Chi phí sở hữu của nhà cung cấp theo số tenant

Hai đường cùng biểu diễn tổng chi phí trong một kỳ. Giao điểm n* là quy mô cân bằng chi phí của hai phương án theo giả thiết; không phải điểm hòa vốn doanh thu hay ngưỡng đã đo của ứng dụng.

Theo giả thiết minh họa, single-tenant có chi phí khởi đầu thấp hơn do cơ chế phục vụ ban đầu đơn giản. Multi-tenant đầu tư sớm cho nhận diện tenant, metadata và cô lập, nhưng chi phí tăng thêm cho mỗi tổ chức thấp hơn nhờ dùng chung tài nguyên. Tại n*, hai phương án có tổng chi phí bằng nhau; trước điểm giao, single-tenant có thể rẻ hơn, sau điểm giao multi-tenant có lợi thế chi phí.

Vị trí n* phụ thuộc tải, yêu cầu cô lập và cách vận hành; có thể không xuất hiện trong phạm vi khảo sát. Đường chi phí dùng chung vẫn tăng khi số tenant tăng. Hình 1.2 là đồ thị khái niệm, không chứa số liệu thực nghiệm, ngưỡng khách hàng thực tế hoặc tỷ lệ tiết kiệm đã đo.

1.2. Các khái niệm và thuật ngữ nền tảng trong kiến trúc Multi-Tenancy

1.2.1. Khách hàng và Thuê bao (Customer & Tenant)

Customer là khách hàng trong quan hệ cung ứng dịch vụ; tenant là đơn vị có phạm vi dữ liệu, cấu hình và quản trị riêng trong hệ thống. Hai khái niệm có thể tương ứng nhưng không bắt buộc một–một. Trong đề tài, một workspace được xác định là một tenant. Giảng viên hoặc nhóm quản lý workspace có thể tổ chức nhiều dự án bên trong mà không tạo tenant mới cho từng dự án.

1.2.2. Người dùng và Thành viên thuê bao (User & Membership)

User là cá nhân có tài khoản truy cập. Membership là quan hệ thành viên giữa user và tenant, kèm vai trò và trạng thái tham gia. Một người có thể thuộc nhiều workspace với quyền khác nhau. user_id xác định ai thực hiện thao tác; tenant_id xác định tổ chức đang được thao tác. Quyền được kiểm tra trên quan hệ membership hiện hành, không suy ra chỉ từ việc đăng nhập thành công.

1.2.3. Dịch vụ và Ứng dụng (Service & Application)

Service là năng lực được cung cấp cho người sử dụng, ví dụ quản lý workspace, quản lý công việc hoặc thông báo. Application là phần mềm hiện thực các năng lực đó. Một ứng dụng có thể chứa nhiều mô-đun nghiệp vụ và cung cấp nhiều dịch vụ qua API. Việc chia trách nhiệm thành các dịch vụ logic không bắt buộc triển khai chúng thành nhiều tiến trình hoặc microservice.

1.2.4. Instance và các ví dụ về Web Server, Application Server, Database Server

Instance là một bản thành phần phần mềm đang chạy và được cấp tài nguyên. Ví dụ, tiến trình Application Server xử lý API Kanban là một instance ứng dụng; tiến trình Web Server phục vụ HTTP và tiến trình Database Server quản lý dữ liệu là các instance có vai trò khác. Báo cáo gọi rõ loại instance khi phân tích để tránh dùng cụm "thực thể vận hành" thiếu ngữ cảnh.

Một máy chủ có thể chạy nhiều instance. Một Database Server có thể quản lý nhiều database. Vì vậy, dùng chung máy hoặc chung Database Server chưa chứng minh các tenant dùng chung ứng dụng hay cùng bảng dữ liệu. Instance ứng dụng trong kiến trúc mục tiêu được hiểu là backend thực thi nghiệp vụ cho nhiều tenant.

1.2.5. Tài nguyên hệ thống (System Resources)

Tài nguyên hệ thống gồm CPU, bộ nhớ, mạng, lưu trữ, kết nối database và các thành phần phần mềm dùng để xử lý yêu cầu. Chia sẻ tài nguyên cần chỉ rõ đối tượng: hai tenant có thể dùng chung backend nhưng database riêng, hoặc chung cả backend lẫn bảng dữ liệu. Chia sẻ hạ tầng không đồng nghĩa chia sẻ quyền đọc, sửa dữ liệu của nhau.

1.2.6. Định danh thuê bao và Siêu dữ liệu (Tenant ID & Metadata)

Tenant ID là định danh của đơn vị thuê bao. Trong phương án bảng chung, các hàng dữ liệu thuộc tenant được gắn định danh này. Metadata là dữ liệu mô tả những biến thể ứng dụng cho phép, chẳng hạn tên và màu workspace, chức năng

được bật hoặc quy tắc WIP. Ứng dụng dùng Tenant ID đã xác minh để chọn đúng dữ liệu và metadata.

Dữ liệu nghiệp vụ và metadata có mục đích khác nhau: sửa nội dung một thẻ công việc là thay đổi dữ liệu; bật một quy tắc làm thay đổi cách ứng dụng xử lý là tùy biến qua metadata. Cả hai đều cần được bảo vệ theo tenant và quyền của người thực hiện.

1.2.7. Định nghĩa Multi-Tenancy và mối quan hệ giữa các khái niệm nền tảng

Kabbedijk và cộng sự diễn giải multi-tenancy là thuộc tính cho phép nhiều khách hàng chia sẻ trong suốt tài nguyên hệ thống nhằm giảm chi phí, đồng thời có khả năng cấu hình theo nhu cầu riêng [3]. Đề tài cụ thể hóa định nghĩa này bằng workspace độc lập về dữ liệu, thành viên và cấu hình trên nền ứng dụng dùng chung.

Quan hệ khái niệm được xác định như sau: khách hàng tạo hoặc quản lý tenant; user tham gia tenant qua membership; dịch vụ được ứng dụng cung cấp; ứng dụng chạy trên instance và sử dụng tài nguyên. Tenant ID nối ngữ cảnh tổ chức với dữ liệu, còn metadata biểu diễn các khác biệt được nền tảng hỗ trợ.

1.3. Phân biệt và mô hình hóa các cách tổ chức hệ thống

1.3.1. Mô hình Đơn thuê bao (Single-Tenant – ST)

Single-Tenant (ST) là cách bố trí tài nguyên tại tầng đang xét dành cho một tenant. Ví dụ, tổ chức A dùng App A và DB A, tổ chức B dùng App B và DB B. Cách tổ chức này thuận lợi cho quản lý biến thể và vận hành độc lập ở các tầng đã tách, nhưng làm tăng số môi trường cần cập nhật. Các instance riêng vẫn có thể nằm trên phần cứng chung.

1.3.2. Mô hình Đa instance (Multi-Instance – MI)

Multi-Instance (MI) mô tả nhiều bản ứng dụng đang chạy. Các instance có thể cùng phục vụ một tenant, mỗi bản phục vụ một tenant, hoặc cùng phục vụ nhiều tenant. Do đó, MI không tự xác định phạm vi cô lập dữ liệu hay mức độ trưởng thành. Khi mô tả MI phải nêu thêm quan hệ tenant–instance và nguồn dữ liệu mà từng instance truy cập.

1.3.3. Mô hình Đa người dùng (Multi-User – MU)

Multi-User (MU) là khả năng cho nhiều cá nhân truy cập và thao tác với danh tính, phiên và quyền tương ứng. Một ứng dụng nội bộ của trường có nhiều tài

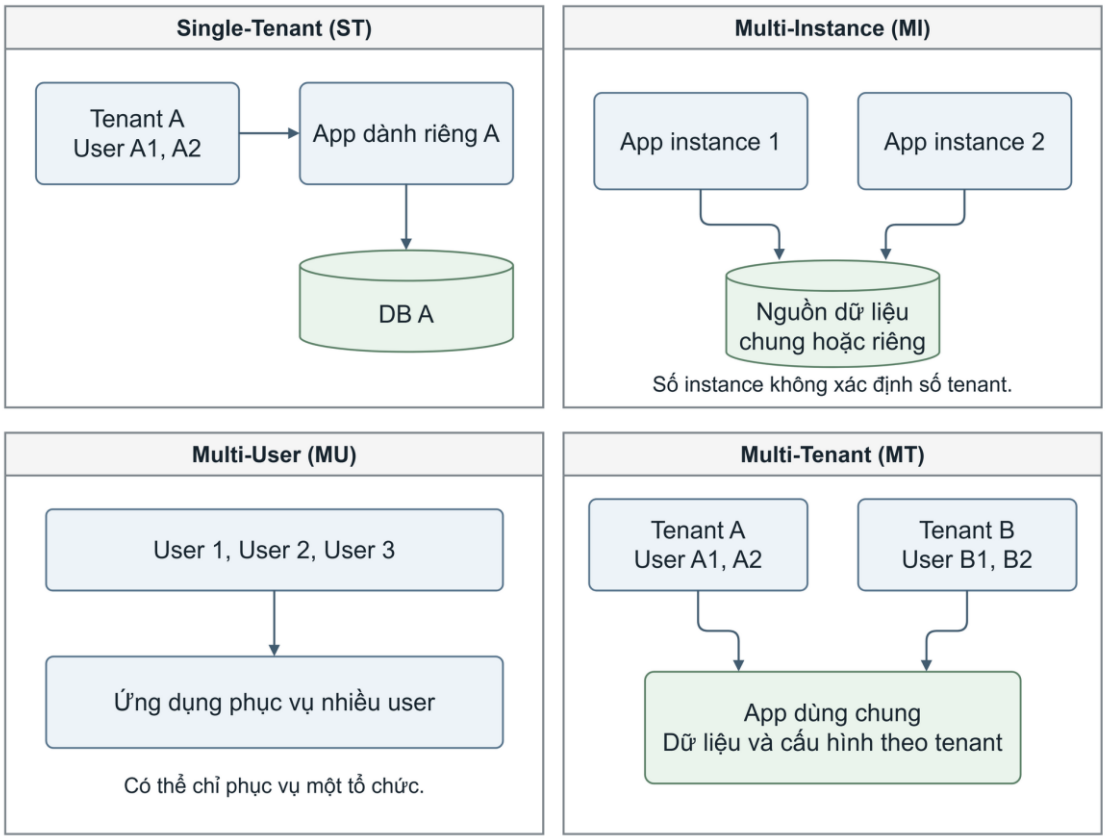
khoản vẫn có thể phục vụ một tenant. MU không mặc nhiên có quản trị phẳng, giao diện cố định hay chỉ một loại quyền; điểm phân biệt là nhiều user chưa tự chứng minh có các tổ chức thuê dịch vụ độc lập.

1.3.4. Mô hình Đa thuê bao (Multi-Tenant – MT)

Multi-Tenant (MT) quản lý nhiều đơn vị thuê bao trên tài nguyên dùng chung, đồng thời phân định dữ liệu, thành viên và cấu hình theo tổ chức. Ví dụ, hai nhóm nghiên cứu cùng sử dụng ứng dụng Kanban nhưng có workspace, quản trị viên và quy tắc riêng. Khả năng cô lập phải được thực thi trong thao tác nghiệp vụ, không chỉ thể hiện bằng hai nhãn workspace trên giao diện [4].

1.3.5. Bảng đối chiếu đa tiêu chí và mô hình hóa trực quan ST, MI, MU và MT

Bốn khái niệm mô tả các phương diện khác nhau và có thể kết hợp. Một hệ thống MT thường đồng thời phục vụ nhiều user; khi mở rộng, nó có thể dùng nhiều instance. Hình 1.3 chỉ rõ đơn vị đang xét và quan hệ với ứng dụng. Bảng 1.1 đối chiếu để tránh dùng số tài khoản hoặc số tiến trình thay cho bằng chứng về quản trị tenant.



Hình 1.3 Phân biệt đơn vị tổ chức của ST MI MU và MT

ST xác định một tenant được phục vụ ở tầng xét; MI đếm các bản đang chạy; MU xác định nhiều cá nhân; MT xác định nhiều tổ chức có phạm vi riêng. Bốn khung là bốn góc nhìn, không phải các bước nâng cấp bắt buộc.

Bảng 1.1 Đối chiếu các cách tổ chức hệ thống

Khái niệm	Đơn vị xét	Quan hệ với ứng dụng
ST	Một tenant	Tài nguyên tầng xét dành cho một tổ chức.
MI	Nhiều instance	Mỗi instance có thể phục vụ một hoặc nhiều tenant.
MU	Nhiều user	Nhiều cá nhân truy cập; có thể cùng một tổ chức.
MT	Nhiều tenant	Dữ liệu, quyền và cấu hình quản lý theo tổ chức.

1.4. Các cấp độ trưởng thành kiến trúc SaaS theo mô hình Chong & Carraro

1.4.1. Cấp độ 1 – Ad Hoc / Custom

Ở mức Ad Hoc / Custom, mỗi khách hàng có bản ứng dụng được tùy biến riêng, có thể phân kỳ mã nguồn, cấu hình và database. Nhà cung cấp phải phối hợp kiểm thử, bảo trì các phiên bản tương ứng. Cách làm này đáp ứng nhu cầu riêng ở quy mô nhỏ nhưng khó duy trì một nền sản phẩm thống nhất khi số khách hàng và số biến thể tăng [2].

1.4.2. Cấp độ 2 – Configurable

Configurable chuẩn hóa một codebase dùng chung. Theo phạm vi đối chiếu của đề tài, mỗi tenant có instance và database riêng, các khác biệt được đưa vào file cấu hình trước khi chạy. App A dùng config A và DB A; App B dùng config B và DB B. Khả năng dùng chung mã giảm phân kỳ phiên bản, nhưng chưa loại bỏ việc vận hành các bản chạy riêng theo khách hàng.

1.4.3. Cấp độ 3 – Configurable, Multi-Tenant-Efficient

Configurable, Multi-Tenant-Efficient là kiến trúc có khả năng cấu hình và phục vụ đa thuê bao hiệu quả. Một instance ứng dụng xử lý nhiều tenant trên cùng mã nguồn; ngữ cảnh tenant quyết định dữ liệu và metadata được áp dụng trong lúc chạy. Phương án của đề tài dùng chung database và bảng nghiệp vụ có Tenant ID để thể hiện rõ bài toán phân tách logic.

Ví dụ, workspace A bật giới hạn WIP, workspace B tắt; cùng mã xử lý đọc metadata và áp dụng quy tắc tương ứng. Thay cấu hình A không tạo nhánh mã, không tạo instance mới và không thay cấu hình B. Quy tắc phải có hiệu lực tại backend, kể cả khi người dùng gọi API trực tiếp.

1.4.4. Cấp độ 4 – Scalable, Configurable, Multi-Tenant-Efficient

Scalable, Configurable, Multi-Tenant-Efficient bổ sung khả năng mở rộng bằng nhiều instance phối hợp, thường có bộ cân bằng tải. Ngữ cảnh tenant, cấu hình và trạng thái phải được quản lý nhất quán khi yêu cầu đi qua các instance khác nhau. Mở rộng dữ liệu là một bài toán liên quan cần thiết kế theo tải.

Container/pod, bản sao dữ liệu hoặc database phân tán là các lựa chọn triển khai theo nhu cầu, không phải tên thành phần bắt buộc của mô hình gốc. Riêng việc đóng gói container hoặc có nhiều bản chạy chưa chứng minh tự động co giãn hay khả năng mở rộng đã được kiểm chứng.

1.4.5. Phân biệt Cấp độ 2 và Cấp độ 3 về cấu hình, instance và cơ sở dữ liệu

Khác biệt quyết định giữa Configurable và Configurable, Multi-Tenant-Efficient là quan hệ tenant–instance và cách áp dụng cấu hình. Phương án thứ nhất gắn cấu hình với từng bản chạy riêng trước khi phục vụ; phương án thứ hai chọn metadata theo tenant ngay trong một ứng dụng đang xử lý nhiều tổ chức. Cùng một repository mã nguồn chưa đủ phân biệt hai cách tổ chức này.

Trong đề tài, phép đối chiếu còn cố định cách lưu trữ: phương án cấu hình riêng dùng DB riêng; kiến trúc mục tiêu dùng DB và bảng chung với Tenant ID. Phạm vi này giúp tổ chức demo và bằng chứng rõ ràng, thay vì coi mọi chiến lược database là điều kiện phổ quát duy nhất của mô hình trưởng thành.

1.4.6. Bảng so sánh bốn cấp độ và định vị kiến trúc mục tiêu của đề tài

Hình 1.4 và Bảng 1.2 tổng hợp sự thay đổi của mã nguồn, cấu hình, instance và dữ liệu. Đề tài định vị ở Configurable, Multi-Tenant-Efficient; trọng tâm kiểm chứng là metadata runtime, một backend phục vụ đồng thời nhiều tenant và database dùng chung được cô lập logic. Mở rộng nhiều instance thuộc hướng phát triển, không được suy ra từ các thử nghiệm trên một backend.



Hình 1.4 Tổ chức mã nguồn ứng dụng và dữ liệu qua bốn cấp độ trưởng thành

Hàng được tô xanh định vị kiến trúc mục tiêu. Điểm phân biệt là metadata được chọn theo tenant trong lúc ứng dụng chung đang chạy; mở rộng thành tập instance là một năng lực bổ sung cần kiểm chứng riêng.

Bảng 1.2 So sánh tổ chức mã nguồn và tài nguyên theo mức trưởng thành

Mức	Mã và cấu hình	App và dữ liệu	Định vị
Ad Hoc / Custom	Biến thể mã theo khách hàng	App và DB riêng	Bối cảnh đối chiếu
Configurable	Codebase chung; config trước khi chạy	App và DB riêng theo tenant	Chuẩn hóa mã
Multi-Tenant-Efficient	Codebase chung; metadata runtime	App chung; bảng chung có Tenant ID	Mục tiêu đề tài

Mức	Mã và cấu hình	App và dữ liệu	Định vị
Scalable Multi-Tenant-Efficient	Kế thừa cấu hình và context tenant	Nhiều App phối hợp; mở rộng dữ liệu	Hướng phát triển

1.5. Mô hình hóa kiến trúc Multi-Tenancy theo ba tầng và ngăn xếp chín tầng

1.5.1. Tầng người thuê – Tổ chức thuê bao và người dùng

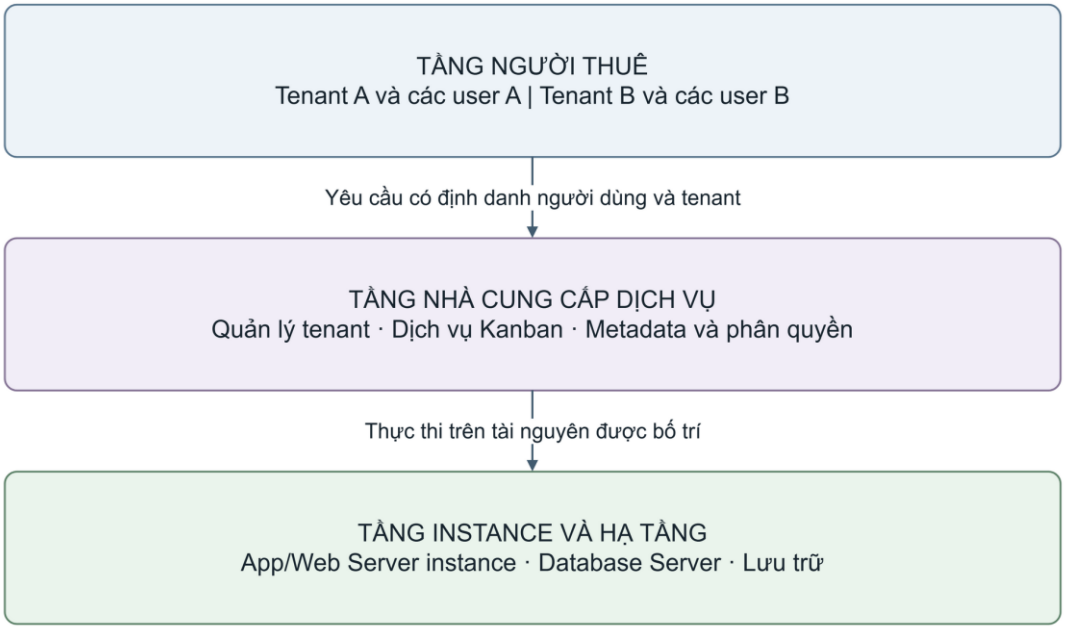
Tầng người thuê gồm các workspace đại diện tenant và các user tham gia qua membership. Quản trị viên workspace quản lý thành viên và các cấu hình được cho phép; thành viên thao tác theo quyền trên dự án, bảng và công việc. Một người có thể tham gia nhiều workspace, nhưng mỗi thao tác phải xác định rõ workspace hiện hành.

1.5.2. Tầng nhà cung cấp dịch vụ – Ứng dụng, nghiệp vụ và cấu hình

Tầng nhà cung cấp dịch vụ hiện thực quản lý tenant, nghiệp vụ Kanban, phân quyền và cấu hình. Tầng này chuyển danh tính user và tenant được chọn thành ngữ cảnh đã xác minh, sau đó áp dụng đúng metadata và phạm vi dữ liệu. Việc dùng chung chức năng không làm mất quyền quản trị riêng của tổ chức trong giới hạn nền tảng hỗ trợ.

1.5.3. Tầng các instance và hạ tầng – Máy chủ ứng dụng và cơ sở dữ liệu

Tầng instance và hạ tầng gồm các bản phần mềm đang chạy như Web/Application Server, Database Server và tài nguyên lưu trữ. Hình 1.5 thể hiện quan hệ cung ứng: tenant sử dụng dịch vụ; dịch vụ được thực thi trên các instance. Đây là cách nhìn theo vai trò, khác với cách chia nội bộ ứng dụng thành lớp giao diện, nghiệp vụ và truy cập dữ liệu.



Hình 1.5 Quan hệ ba tầng trong cung ứng dịch vụ đa thuê bao

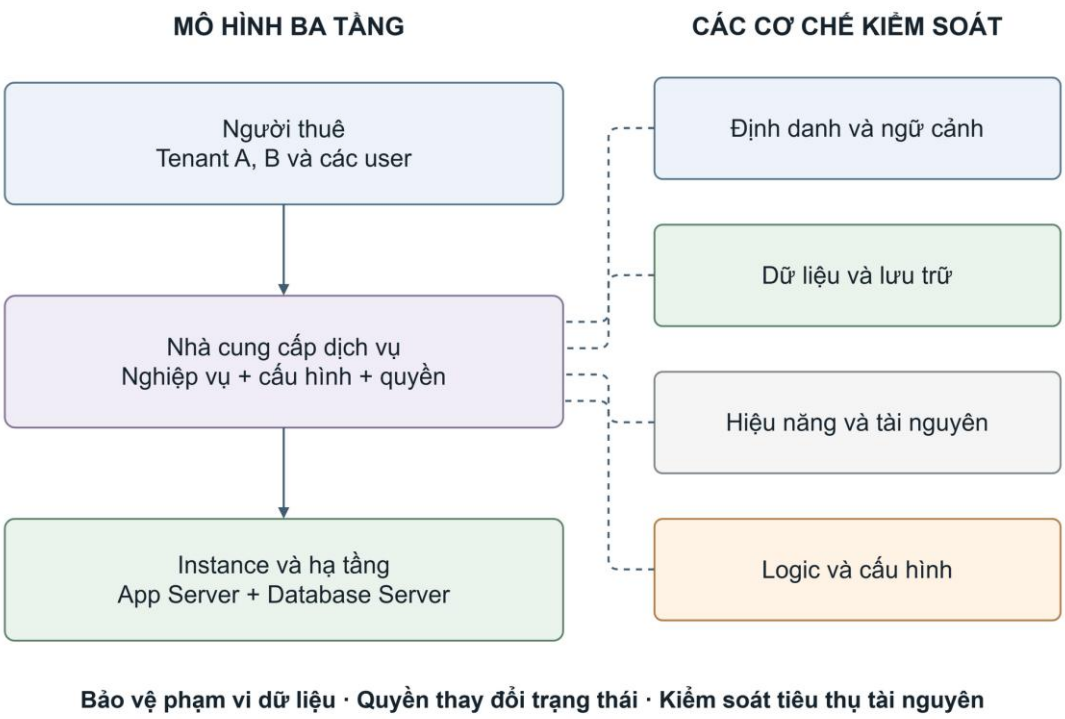
Mũi tên nối chủ thể sử dụng, dịch vụ và môi trường thực thi. Web/Application Server và Database Server là ví dụ instance cụ thể; một tenant không được đồng nhất với một máy chủ hay một tài khoản.

1.5.4. Quan hệ chia sẻ và tách biệt tài nguyên qua bốn cấp độ trưởng thành

Qua các mức trưởng thành, tầng người thuê vẫn có các tổ chức và user, nhưng cách bố trí tài nguyên thay đổi. Ad Hoc / Custom và phương án Configurable đối chiếu sử dụng môi trường riêng theo khách hàng. Configurable, Multi-Tenant-Efficient dùng chung bản chạy với kiểm soát theo tenant; kiến trúc mở rộng tổ chức nhiều bản chạy phối hợp. Mô hình ba tầng giúp giữ nhất quán chủ thể sử dụng, chức năng và đơn vị thực thi khi so sánh.

1.5.5. Mô hình hóa tổng thể kiến trúc đa thuê bao và các cơ chế kiểm soát xuyên tầng

Hình 1.6 tích hợp mô hình ba tầng với bốn nhóm kiểm soát: định danh và ngữ cảnh; dữ liệu và lưu trữ; hiệu năng và tài nguyên; logic và cấu hình. Ba mục tiêu bảo vệ là phạm vi nhìn thấy dữ liệu, quyền thay đổi trạng thái và mức tiêu thụ tài nguyên. Đây là các phương diện phân tích, không phải một thang cấp độ trưởng thành khác.



Hình 1.6 Mô hình tổng thể và các cơ chế kiểm soát xuyên tầng

Các đường nét đứt chỉ quan hệ áp dụng kiểm soát, không biểu diễn đường đi mạng hoặc các dịch vụ phải triển khai riêng. Cơ chế tại ứng dụng phối hợp với database và tài nguyên để duy trì phạm vi thuê bao.

Đường liền thể hiện quan hệ phục vụ giữa các tầng; nét đứt thể hiện phạm vi áp dụng kiểm soát. Các nhóm kiểm soát không phải những máy chủ trung gian mà request phải lần lượt đi qua. Ví dụ, cùng backend vừa xác minh TenantContext, áp dụng metadata vừa giới hạn thao tác database; các trách nhiệm này phối hợp trong một luồng nghiệp vụ.

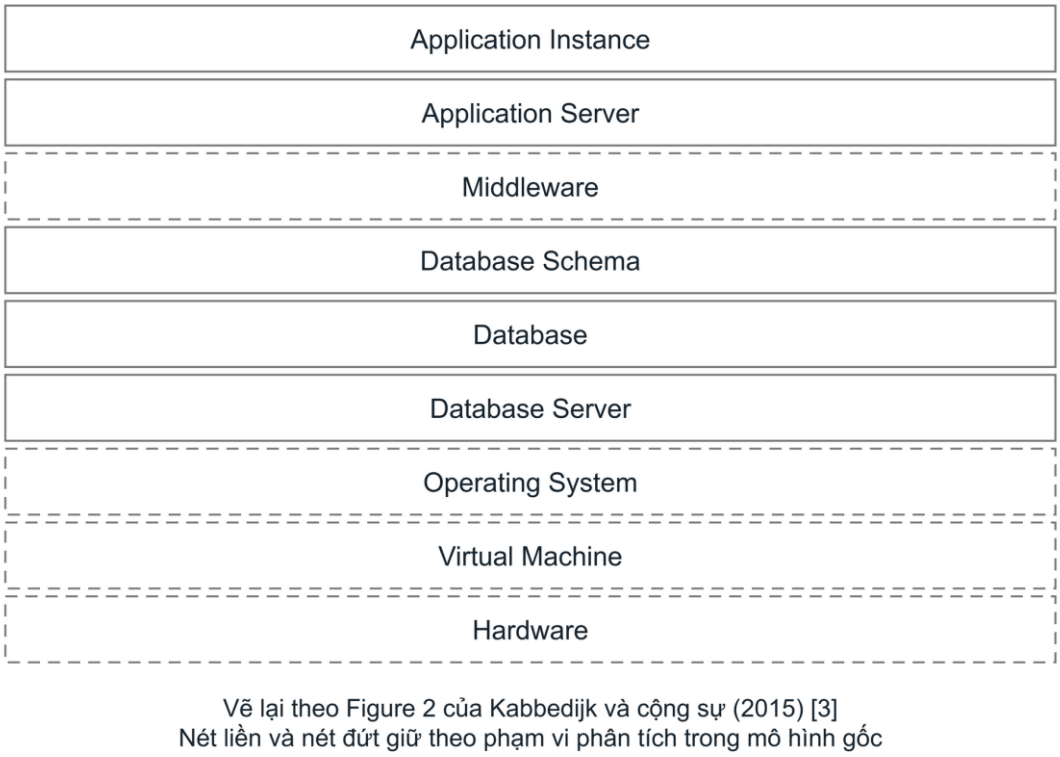
1.5.6. Ánh xạ mô hình tổng thể vào kiến trúc mục tiêu Cấp độ 3

Ánh xạ vào kiến trúc mục tiêu, hai tenant A và B cùng gửi yêu cầu đến một backend và cùng database. Xác thực và membership tạo ngữ cảnh tin cậy; metadata chọn biến thể giao diện, nghiệp vụ; Tenant ID cùng chính sách truy cập giới hạn dữ liệu. Đối chiếu thiết kế phải chỉ ra cả phần dùng chung lẫn điểm kiểm soát giữ riêng, thay vì chỉ bao quanh sơ đồ bằng một khung “tenant boundary”.

1.5.7. Ngăn xếp chín tầng nguyên bản của Kabbedijk và cộng sự

Bên cạnh cách nhìn theo chủ thể và vai trò ở mô hình ba tầng, Kabbedijk và cộng sự mô tả các thành phần kỹ thuật bằng ngăn xếp chín tầng trong Figure 2 [3]. Hình 1.7 vẽ lại mô hình nguyên bản, giữ thứ tự thành phần và cách phân biệt nét

liền, nét đứt. Từ dưới lên là phần cứng, máy ảo, hệ điều hành, máy chủ cơ sở dữ liệu, database, schema, middleware, máy chủ ứng dụng và instance ứng dụng.



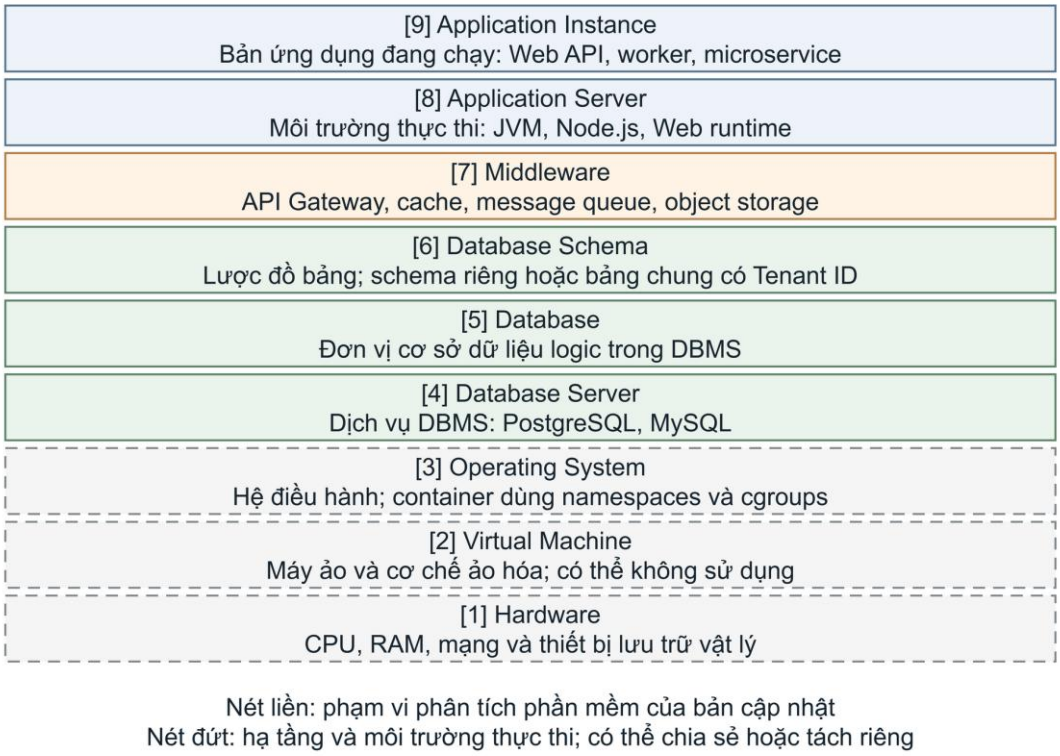
Hình 1.7 Ngăn xếp chín tầng nguyên bản của Kabbedijk và cộng sự (vẽ lại theo [3])

Bản không màu giữ quy ước của mô hình gốc: năm khối nét liền và bốn khối nét đứt. Mỗi khối là một mức thành phần kỹ thuật; thứ tự không biểu diễn luồng gọi API hay số máy chủ phải triển khai.

Trong phạm vi phân tích của bài báo gốc, năm khối nét liền là Database Server, Database, Database Schema, Application Server và Application Instance. Middleware cùng ba tầng hạ tầng được vẽ nét đứt; tác giả có ghi nhận nghiên cứu đa thuê bao ở middleware nhưng không phân tích riêng để giữ định nghĩa tổng quát [3]. Vì vậy, không diễn giải nét đứt thành kết luận rằng các thành phần này không thể chia sẻ hoặc cô lập theo tenant.

1.5.8. Ngăn xếp chín tầng cập nhật cho kiến trúc Cloud-Native

Hình 1.8 là bản cập nhật do nhóm đề xuất trên cấu trúc chín tầng, phân biệt với mô hình nguyên bản ở Hình 1.7. Điều chỉnh chính là đưa Middleware vào nhóm nét liền để phân tích các điểm kiểm soát tenant tại API Gateway, cache, hàng đợi và lưu trữ đối tượng khi chúng xuất hiện trong hệ thống. Ví dụ, cache cần khóa có tenant_id và tác vụ trong hàng đợi cần mang ngữ cảnh tenant đã được xác minh.



Hình 1.8 Ngăn xếp chín tầng cập nhật cho kiến trúc Cloud-Native do nhóm đề xuất

Màu xanh dương chỉ ứng dụng; xanh lá chỉ nhóm dữ liệu; cam nhấn mạnh Middleware được chuyển sang nét liền trong bản cập nhật. Ba khối hạ tầng phía dưới vẫn dùng nét đứt để phân biệt phạm vi phân tích, không biểu thị mức an toàn thấp hơn.

Các tầng 4–9 là trọng tâm phân tích phần mềm; tầng 1–3 cung cấp hạ tầng và môi trường thực thi. Container được chú thích tại hệ điều hành vì dùng cơ chế namespaces và cgroups trên nhân dùng chung, không thêm thành tầng thứ mười. Container có thể chạy trên máy ảo hoặc máy vật lý; nét đứt không bắt buộc hạ tầng phải dùng chung và không loại bỏ vai trò cô lập tài nguyên.

Hai mô hình bổ trợ nhau: ba tầng trả lời ai sử dụng, ai cung cấp và dịch vụ chạy ở đâu; chín tầng chỉ rõ thành phần kỹ thuật được chia sẻ hoặc tách riêng. Trong thiết kế đề tài, nhiều tenant dùng cùng backend và các bảng có Tenant ID; TenantContext, metadata và kiểm soát dữ liệu giữ phạm vi riêng. Application Server chỉ môi trường thực thi, Application Instance chỉ bản ứng dụng đang chạy; chúng có thể cùng nằm trong một tiến trình backend. Chín tầng là khung phân tích, không yêu cầu triển khai chín máy chủ hoặc chứng minh khả năng mở rộng ngang.

1.6. Các đặc trưng kiến trúc cốt lõi của hệ thống Multi-Tenancy

1.6.1. Chia sẻ tài nguyên trong suốt (Transparent Resource Sharing)

Chia sẻ trong suốt cho phép người dùng làm việc trong workspace mà không cần biết yêu cầu đang được thực thi trên tài nguyên chung nào. Hệ thống che giấu cách cấp phát nhưng vẫn giữ ý nghĩa dữ liệu và quyền của tổ chức. Trong suốt không đồng nghĩa độc lập hoàn toàn về hiệu năng; cạnh tranh CPU, kết nối hoặc I/O vẫn có thể xuất hiện.

1.6.2. Tùy biến động lúc chạy dựa trên siêu dữ liệu (Metadata-Driven Runtime Customization)

Tùy biến runtime dùng metadata để biểu diễn những điểm biến đổi đã được mã nguồn hỗ trợ. Cấu hình được chọn theo tenant trong lúc xử lý, được kiểm tra hợp lệ và cập nhật đúng phạm vi. Tên, màu workspace và quy tắc WIP là ví dụ trong đề tài. Tùy biến không bao gồm quyền thực thi mã tùy ý của khách hàng hoặc sửa trực tiếp database.

1.6.3. Cô lập dữ liệu, ngữ cảnh và tài nguyên giữa các thuê bao (Tenant Isolation)

Cô lập bao gồm kiểm soát dữ liệu, ngữ cảnh và ảnh hưởng của tài nguyên dùng chung. Tenant không được đọc hoặc thay đổi tài nguyên của tổ chức khác nếu không có quyền hợp lệ. Rò ngữ cảnh trên thread, connection hoặc cache cũng có thể làm sai phạm vi dù dữ liệu đã mang Tenant ID. Cô lập dữ liệu và cô lập hiệu năng cần được đánh giá bằng các tình huống riêng.

1.6.4. Khả năng mở rộng, tự phục vụ và quản lý vòng đời thuê bao (Scalability, Self-Service & Tenant Lifecycle Management)

Quản lý vòng đời gồm khởi tạo, cấu hình, tạm ngưng và kết thúc sử dụng tenant. Tự phục vụ có thể giảm thao tác của nhà cung cấp; mức tự động hóa phụ thuộc phạm vi ứng dụng. Khả năng mở rộng là điều chỉnh năng lực theo tải, nhưng demo một instance không chứng minh mở rộng ngang. Đề tài ưu tiên quy trình khởi tạo nhất quán và chỉ đặt mở rộng ngang ở hướng phát triển.

1.6.5. Đo lường và giám sát theo thuê bao (Tenant-Aware Metering & Monitoring)

Đo lường theo tenant gắn yêu cầu và chỉ số với tổ chức tương ứng để quan sát độ trễ, lỗi và mức sử dụng. Log cần có request_id, tenant_id đã xác minh và instance_id để truy nguyên. Giám sát hỗ trợ phát hiện bất thường nhưng không

thay thế cơ chế hạn mức hoặc phân quyền. Thu thập mức sử dụng cũng khác với xây dựng một hệ thống tính cước thương mại.

1.6.6. Liên hệ các đặc trưng kiến trúc với tiêu chí kiểm chứng Cấp độ 3

Các đặc trưng trên được chuyển thành tiêu chí thiết kế và kiểm chứng ở Bảng 1.3. Ba nhóm bằng chứng trực tiếp của kiến trúc mục tiêu là mã chung với metadata runtime, một instance phục vụ nhiều tenant và dữ liệu chung có cô lập logic. Vòng đời, quan sát và điều tiết tài nguyên hỗ trợ tính khả thi vận hành; chúng không tự thay thế các nhóm bằng chứng này.

Bảng 1.3 Liên hệ đặc trưng với nội dung kiểm chứng

Đặc trưng	Biểu hiện trong thiết kế	Bằng chứng cần có
Chia sẻ	App và database chung	Cấu hình triển khai, instance_id và lược đồ.
Tùy biến runtime	Metadata riêng theo tenant	Đổi cấu hình A lúc chạy; B giữ nguyên.
Cô lập	Context, quyền và Tenant ID	Thử đọc/ghi chéo, thiếu tenant và nhánh lỗi.
Vòng đời	Khởi tạo tenant bằng dữ liệu	Kiểm tra tạo đầy đủ hoặc rollback.
Quan sát	Chỉ số và log theo tenant	Request, tenant và instance nối được với thao tác.

1.7. Tổng kết chương

Chương 1 xác lập quan hệ tenant–user–membership, phân biệt các cách tổ chức hệ thống và sử dụng mô hình trưởng thành SaaS để định vị kiến trúc mục tiêu. Mô hình ba tầng cùng các phương diện kiểm soát giúp mô tả phần dùng chung và phạm vi riêng. Chương 2 tiếp tục so sánh các phương án triển khai và đánh đổi trước khi lựa chọn cấu hình phù hợp cho ứng dụng Kanban.

CHƯƠNG 2. KIẾN TRÚC ĐA THUÊ BAO — CƠ CHẾ CHIA SẺ, CÔ LẬP VÀ CÁC ĐÁNH ĐỔI

Chương này phân tích không gian giải pháp từ các mô hình triển khai đến cơ chế chia sẻ và cô lập. Các đánh đổi được xem xét trước khi lựa chọn kiến trúc phù hợp với ứng dụng quản lý công việc, tránh đồng nhất mọi phương án đa thuê bao với cùng một cấu hình triển khai.

2.1. Nguyên lý tổ chức kiến trúc đa thuê bao

2.1.1. Chia sẻ và tách biệt tài nguyên

Chia sẻ và tách biệt là hai thao tác bố trí tài nguyên tại một tầng cụ thể. Chia sẻ có thể giảm số thành phần phải cấp phát và cập nhật; tách biệt tạo đơn vị quản lý riêng nhưng tăng công việc vận hành. Một hệ thống có thể dùng chung backend và tách database, hoặc tách backend nhưng dùng chung dịch vụ hỗ trợ. Vì vậy, mô tả kiến trúc phải nêu rõ đối tượng được chia sẻ.

2.1.2. Ranh giới thuê bao và các phương diện cô lập

Ranh giới thuê bao được thực thi tại các điểm xác minh ngữ cảnh, phân quyền, truy cập dữ liệu, áp dụng cấu hình và cấp tài nguyên. Đăng nhập chỉ xác định danh tính, chưa bảo đảm mọi thao tác nằm trong tenant hợp lệ. Cô lập phải được duy trì xuyên suốt luồng xử lý, kể cả khi truy vấn qua ID, xử lý nền hoặc tái sử dụng kết nối.

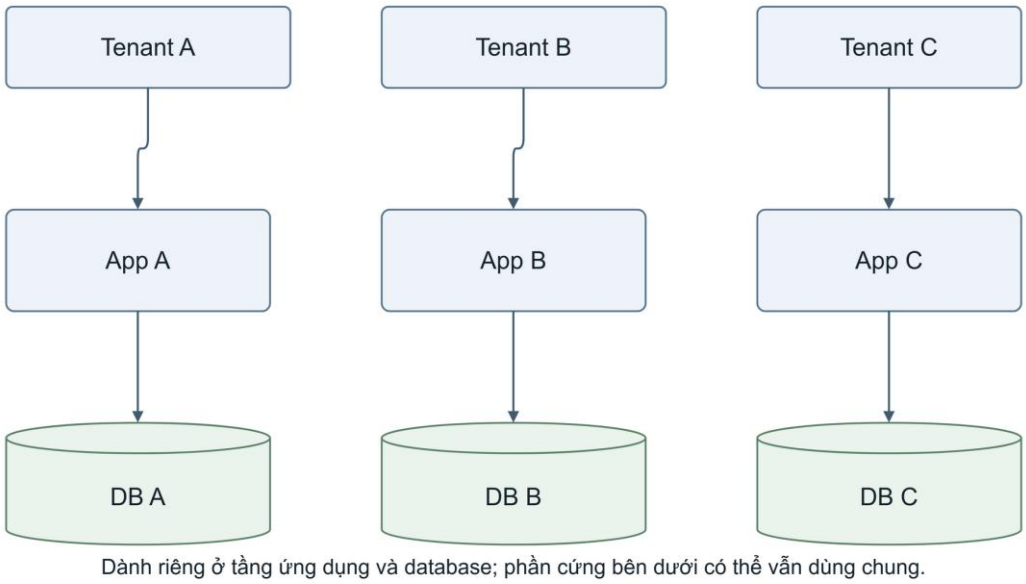
2.1.3. Quan hệ giữa mô hình triển khai và mức độ trưởng thành SaaS

Silo, Pool, Bridge và Hybrid mô tả cách bố trí tài nguyên; mô hình Chong & Carraro phân tích sự trưởng thành về cấu hình, phục vụ nhiều tenant và mở rộng. Hai khung liên quan nhưng không tương ứng cứng từng mức. Ví dụ, cùng mã nguồn có thể được triển khai trong nhiều instance riêng hoặc một backend chung. Việc định vị cần đồng thời xem mã, cấu hình, đơn vị thực thi và dữ liệu.

2.2. Các mô hình triển khai kiến trúc đa thuê bao

2.2.1. Mô hình Silo

Silo bố trí tài nguyên dành riêng theo tenant ở những tầng được xác định. Hình 2.1 minh họa App và DB riêng cho A, B, C. Ưu điểm là xác định rõ đơn vị cấu hình, nâng cấp và sao lưu. Hạn chế là tăng số môi trường và có thể để tài nguyên nhàn rỗi. Tách tiến trình hoặc database không mặc nhiên đồng nghĩa tách phần cứng vật lý.

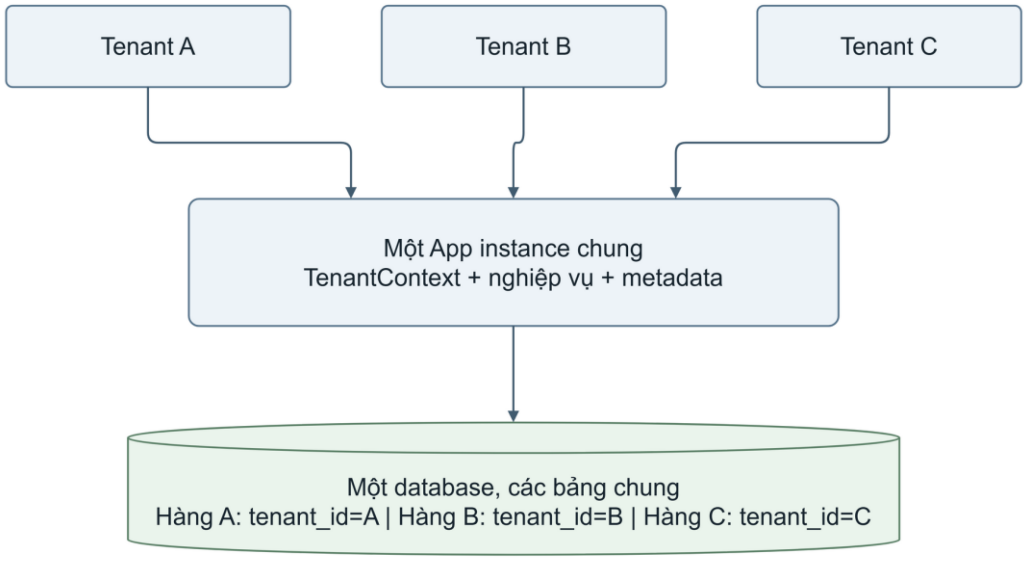


Hình 2.1 Tài nguyên ứng dụng và dữ liệu dành riêng theo tenant

Mỗi tenant đi vào đúng App và database của mình. Sơ đồ chỉ xác nhận tách ở hai tầng được vẽ; nó không khẳng định các máy vật lý bên dưới cũng dành riêng.

2.2.2. Mô hình Pool

Pool tổ chức một tập tài nguyên chung phục vụ nhiều tenant. Trong Hình 2.2, backend và các bảng chung sử dụng TenantContext và Tenant ID để xác định phạm vi dữ liệu. Cách làm này thuận lợi cho khởi tạo tenant bằng dữ liệu và cập nhật tập trung, nhưng phụ thuộc vào tính đúng đắn của phần mềm kiểm soát. Lỗi dùng nhầm context hoặc thiếu giới hạn truy cập có thể tác động nhiều tổ chức.

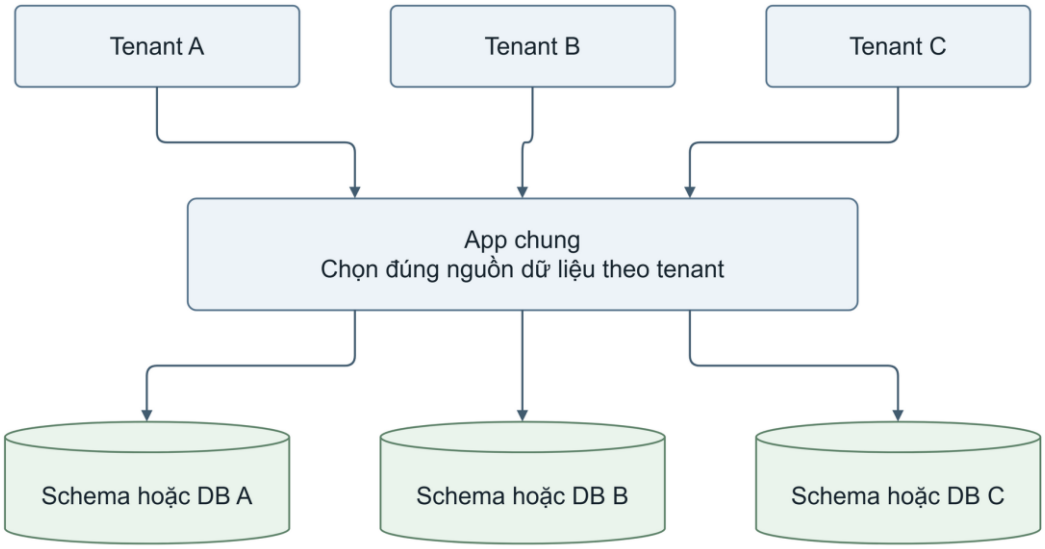


Hình 2.2 Ứng dụng và bảng dữ liệu dùng chung có nhận biết tenant

A, B, C cùng đi vào một App và bảng dữ liệu chung. TenantContext chọn cấu hình và phạm vi xử lý; tenant_id phân biệt hàng dữ liệu, còn chính sách truy cập thực thi quyền.

2.2.3. Mô hình Bridge

Bridge phối hợp tài nguyên dùng chung và dành riêng ở các tầng khác nhau [5]. Hình 2.3 chọn ví dụ một App chung với schema hoặc database riêng theo tenant. Ứng dụng phải ánh xạ đúng tenant sang nguồn dữ liệu trước khi xử lý. Phân tách dữ liệu có thể hỗ trợ cấu hình hoặc sao lưu độc lập, đổi lại cần quản lý nhiều đơn vị dữ liệu và theo dõi việc cập nhật lược đồ.

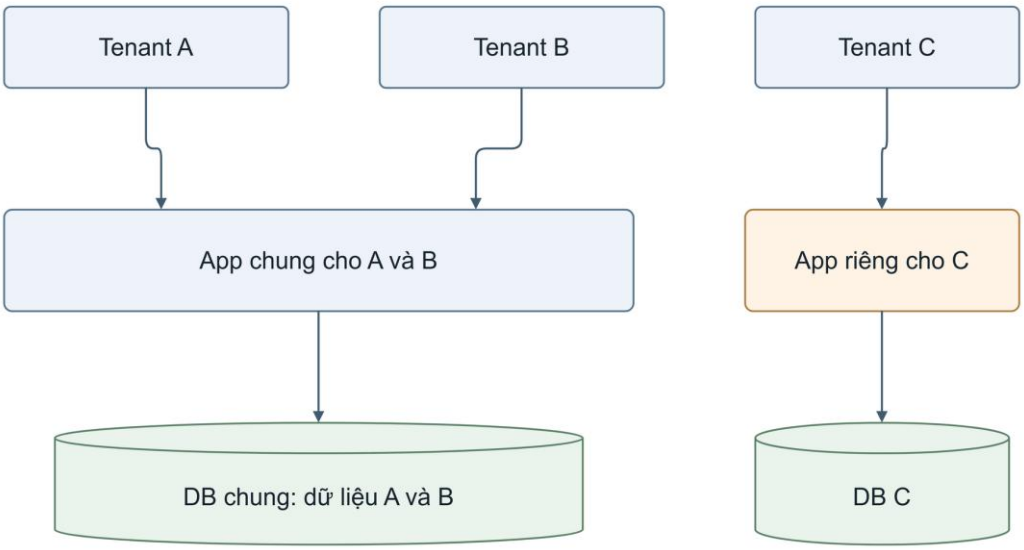


Hình 2.3 Chia sẻ ứng dụng và phân tách nguồn dữ liệu

Backend dùng chung chọn schema hoặc database tương ứng tenant đã xác minh. Hai cách lưu trữ là các lựa chọn thay thế trong ví dụ, không phải yêu cầu cùng tạo cả schema lẫn database riêng.

2.2.4. Mô hình Hybrid và các biến thể mở rộng

Hybrid trong báo cáo là cách phối hợp nhiều phương án theo nhóm tenant hoặc yêu cầu dịch vụ. Hình 2.4 minh họa A, B dùng Pool còn C có App và DB riêng. Các biến thể như Compute-Isolated / Shared-Data tách môi trường xử lý nhưng giữ chung dữ liệu; Cell-Based chia hệ thống thành các đơn vị phục vụ nhóm tenant. Những lựa chọn này cần cơ chế ánh xạ tenant, quản lý cấu hình và xử lý thay đổi vị trí phục vụ.



Hình 2.4 Phối hợp tài nguyên dùng chung và dành riêng

A và B dùng cùng App, database; C được bố trí nguồn riêng. Phương án này phát sinh trách nhiệm quản lý ánh xạ tenant và cập nhật đồng bộ giữa các nhóm triển khai.

2.2.5. So sánh đặc điểm và điều kiện áp dụng

Bảng 2.1 đối chiếu các mô hình theo phạm vi dùng chung, điểm kiểm soát và điều kiện sử dụng. Đánh giá mang tính định tính, không phải xếp hạng hay kết quả đo. Silo không tự bảo đảm an toàn tuyệt đối; Pool không mặc nhiên không an toàn. Lựa chọn cuối cùng phụ thuộc yêu cầu dữ liệu, tùy biến, tải và khả năng vận hành của nhóm.

Bảng 2.1 So sánh các mô hình triển khai

Mô hình	Phạm vi minh họa	Thuận lợi	Đánh đổi
Silo	App, DB riêng	Quản lý môi trường độc lập	Nhiều đơn vị cấp phát và bảo trì.
Pool	App, bảng chung	Chia sẻ tài nguyên và cập nhật	Phụ thuộc kiểm soát logic; cạnh tranh tài nguyên.
Bridge	App chung; dữ liệu tách	Thêm đơn vị quản lý dữ liệu	Định tuyến và cập nhật nhiều nguồn.
Hybrid	Kết hợp theo nhóm tenant	Đáp ứng nhu cầu khác nhau	Tăng cấu hình, điều phối và vận hành.

2.3. Các cơ chế chia sẻ và cô lập

2.3.1. Tùy biến giao diện và nghiệp vụ bằng metadata

Ứng dụng có thể dùng mã chung với các điểm biến đổi được mô tả bằng metadata. Giao diện chọn tên, màu hoặc thành phần hiển thị theo tenant; backend áp dụng các quy tắc nghiệp vụ tương ứng. Feature flag thể hiện một dạng điều kiện bật/tắt, nhưng không thay thế kiểm tra quyền. Khi tắt chức năng trên giao diện, API vẫn phải xử lý theo quyền và cấu hình đã xác minh.

2.3.2. Tổ chức và cô lập dữ liệu

Ba phương án phổ biến là database riêng, schema riêng và bảng chung có Tenant ID. Bảng 2.2 phân biệt đơn vị phân tách và công việc quản lý tương ứng. Đề tài chọn bảng chung để thể hiện trực tiếp bài toán nhiều tenant trên cùng nguồn dữ liệu, đồng thời thiết kế ràng buộc quan hệ và kiểm soát truy cập theo tổ chức.

Với PostgreSQL, RLS bổ sung giới hạn hàng dữ liệu tại DBMS. Thiết kế xét điều kiện đọc hoặc sửa hàng và điều kiện đối với dữ liệu mới; tài khoản ứng dụng không được có quyền vượt chính sách. RLS phụ thuộc vào tenant context đúng và không thay thế phân quyền nghiệp vụ. Cấu hình vai trò và hiệu lực chính sách phải được kiểm chứng bằng tài khoản thực tế [7].

Bảng 2.2 Đối chiếu các phương án lưu trữ

Phương án	Đơn vị tách	Trách nhiệm vận hành
Database-per-Tenant	Database riêng	Quản lý kết nối, migration và sao lưu nhiều DB.
Schema-per-Tenant	Nhóm bảng/schema riêng	Chọn đúng schema; quản lý nhiều nhóm bảng.
Shared-Database / Shared-Schema	Bảng chung có tenant_id	Thực thi quyền và ràng buộc dữ liệu theo tenant.

2.3.3. Quản lý ngữ cảnh và quyền truy cập theo tenant

Tenant được chọn qua đường dẫn, subdomain hoặc header phải được đối chiếu với membership của user đã xác thực. TenantContext sau xác minh được dùng nhất quán trong nghiệp vụ, metadata và dữ liệu. Không có quyền hoặc thiếu tenant thì từ chối trước khi thao tác. Các định danh đối tượng do client gửi phải được kiểm tra trong phạm vi tenant, không truy vấn chỉ bằng object_id.

Ngữ cảnh phải có vòng đời xác định. Nếu backend dùng thread-local thì giải phóng trong finally; tác vụ nền phải thiết lập và dọn context riêng. Khi đưa context vào PostgreSQL bằng SET LOCAL, lệnh thiết lập và truy vấn nghiệp vụ phải ở cùng transaction. Sau khi transaction kết thúc, trạng thái session trước đó có thể trở lại, nên không được duy trì tenant sai ở mức session [8].

2.3.4. Phân bổ tài nguyên và hạn chế ảnh hưởng giữa các tenant

Tài nguyên dành riêng tạo đơn vị giới hạn rõ hơn nhưng có thể phân mảnh năng lực. Pool tận dụng tài nguyên chung, đồng thời có nguy cơ một tenant gây chờ đợi cho các tenant khác. Điều tiết request, giới hạn đồng thời, timeout và theo dõi kết nối giúp quản lý tải. Những cơ chế này cần được đánh giá dưới workload xác định; rate limiting riêng lẻ không trực tiếp bảo đảm giới hạn CPU hoặc RAM.

Cache, tệp và tác vụ nền cũng cần phạm vi tenant. Khóa cache hoặc khóa lưu trữ gồm tenant_id và định danh tài nguyên; worker xác minh nguồn tác vụ trước khi áp dụng ngữ cảnh. Tách tên khóa ngăn dùng nhầm dữ liệu nhưng chưa tạo hạn mức dung lượng. Cơ chế chỉ đưa vào thiết kế khi có chức năng tương ứng trong ứng dụng.

2.3.5. Đánh đổi về chi phí, khả năng tùy biến và vận hành

Đánh đổi cần xét đồng thời chi phí khởi đầu, chi phí tăng thêm, độ tự do tùy biến và công sức vận hành. Môi trường riêng thuận lợi cho thay đổi đặc thù nhưng tăng số đơn vị bảo trì; môi trường chung giảm cấp phát lặp lại nhưng đòi hỏi kiểm soát phần mềm nhất quán. Phân tách logic không tự quyết định mức chi phí hay thời gian khởi tạo.

Quan hệ giữa mức phân tách và chi phí phụ thuộc tải, công nghệ và yêu cầu chất lượng dịch vụ. Các phương án cần được so sánh trên cùng nhu cầu với giả thiết được nêu rõ, như đồ thị TCO ở Mục 1.1.4.

2.4. Các kịch bản kiến trúc và lựa chọn phương án

2.4.1. Kịch bản ưu tiên chi phí và khởi tạo nhanh

Kịch bản ưu tiên chi phí và khởi tạo nhanh phù hợp với nhóm tenant có nghiệp vụ tương đối đồng nhất, khác biệt có thể biểu diễn bằng cấu hình. Pool cho phép thêm tenant qua danh mục, membership và metadata thay vì tạo môi trường mới. Đổi lại, dữ liệu chung phải được kiểm soát và cần quan sát tải theo tenant. Đây là giả thiết lựa chọn, không phải cam kết khởi tạo tức thì.

2.4.2. Kịch bản ưu tiên cô lập và vận hành độc lập

Kịch bản ưu tiên cô lập và vận hành độc lập xem xét Silo hoặc Bridge tại các tầng liên quan. Database riêng có thể tạo đơn vị sao lưu và lịch nâng cấp độc lập hơn, nhưng không loại bỏ trách nhiệm xác thực, định tuyến và bảo vệ dữ liệu. Việc tách tài nguyên được lựa chọn từ yêu cầu cụ thể; không mặc định một tên tiêu chuẩn hay phân khúc khách hàng luôn yêu cầu tách toàn bộ hạ tầng.

2.4.3. Kịch bản phục vụ nhiều phân khúc thuê bao

Khi các nhóm tenant khác nhau về hạn mức, tùy biến hoặc mức độc lập vận hành, Hybrid có thể phối hợp Pool với nguồn tài nguyên riêng. Nhà cung cấp cần quản lý tenant nằm ở đâu, áp dụng cấu hình nào và cách chuyển dữ liệu khi thay đổi phương án. Cell-Based là hướng bổ sung khi có nhu cầu phân nhóm tải hoặc phạm vi sự cố, không phải thành phần bắt buộc của mọi nền tảng nhiều phân khúc.

2.4.4. So sánh đánh đổi và căn cứ lựa chọn

Bảng 2.3 tổng hợp các kịch bản và điều kiện đánh đổi. Ứng dụng trong thuyết minh phục vụ giảng viên, sinh viên và nhóm dự án ở quy mô vừa phải, chưa đặt yêu cầu môi trường riêng cho từng khách hàng. Nghiệp vụ Kanban thống nhất và tùy biến có giới hạn là căn cứ chọn backend, database dùng chung. Các phương án còn lại được giữ như cơ sở so sánh và hướng phát triển.

Bảng 2.3 So sánh các kịch bản kiến trúc

Ưu tiên	Phương án xem xét	Điều kiện và đánh đổi
Chi phí, khởi tạo nhanh	Pool	Nghiệp vụ tương đối đồng nhất; kiểm soát dữ liệu chung.
Vận hành độc lập	Silo hoặc Bridge	Cần đơn vị dữ liệu/môi trường riêng; tăng chi phí quản lý.
Nhiều phân khúc	Hybrid; Cell khi cần	Quản lý vị trí tenant và chính sách khác nhau; tăng điều phối.

2.5. Kiến trúc có khả năng cấu hình và phục vụ đa thuê bao hiệu quả (Configurable, Multi-Tenant-Efficient)

2.5.1. Mô hình tổng thể và phạm vi áp dụng cho đề tài

Kiến trúc Configurable, Multi-Tenant-Efficient được chọn để cung cấp nhiều workspace trên một nền phần mềm thống nhất. Mỗi workspace là tenant với

thành viên, dữ liệu và metadata riêng. Thiết kế tập trung vào chức năng quản lý công việc, cô lập logic và cấu hình lúc chạy; không mở rộng sang tính cước thương mại, chạy mã tùy ý hay vận hành hạ tầng phân tán quy mô lớn.

2.5.2. Mã nguồn chung và instance ứng dụng dùng chung

Một codebase hiện thực các chức năng dùng chung; một backend instance tiếp nhận yêu cầu của nhiều workspace. Các mô-đun xác thực, tenant, Kanban và cấu hình cùng hoạt động trong backend, thay vì tạo bản chạy theo tổ chức. Thêm tenant là khởi tạo định danh, quyền thành viên và cấu hình cần thiết. Danh tính instance và phiên bản mã được giữ trong hồ sơ triển khai để kiểm chứng việc chia sẻ thực tế.

2.5.3. Cơ sở dữ liệu dùng chung và phân tách bằng Tenant ID

Database chung lưu các bảng nghiệp vụ theo Tenant ID. Truy vấn giới hạn theo tenant và quyền, ràng buộc dữ liệu ngăn đối tượng con trỏ sang tài nguyên của tổ chức khác. Việc dùng chung không cho phép quản trị viên tenant truy cập dữ liệu nền tảng. Chương 3 cụ thể hóa khóa, quan hệ và phạm vi truy cập để nguyên tắc này có thể hiện thực bằng migration và mã nghiệp vụ.

2.5.4. Tùy biến runtime qua metadata và luồng xử lý theo tenant

Mỗi request đi qua xác thực và membership, nhận TenantContext đã xác minh, chọn metadata rồi thực hiện nghiệp vụ trong phạm vi dữ liệu tương ứng. Thay cấu hình workspace A làm thay đổi biến thể được hỗ trợ cho A mà không sửa mã hoặc khởi động lại backend. Workspace B giữ cấu hình của mình. Luồng này liên kết việc chia sẻ instance với khả năng tùy biến và tính độc lập của từng tenant.

2.6. Đánh giá và hướng phát triển kiến trúc

2.6.1. Tiêu chí và phương pháp kiểm chứng kiến trúc được lựa chọn

Việc đánh giá đối chiếu ba nhóm thuộc tính ở Bảng 2.4: mã chung và metadata runtime, một instance phục vụ đồng thời nhiều tenant, database chung có phân tách logic và ngữ cảnh đúng. Bằng chứng gồm phiên bản mã, cấu hình triển khai, log request, lược đồ dữ liệu và phản hồi của các ca truy cập hợp lệ hoặc bị từ chối. Sơ đồ thiết kế và ảnh giao diện riêng lẻ chưa đủ kết luận các thuộc tính này đã được đáp ứng.

Bảng 2.4 Các nhóm bằng chứng của kiến trúc được lựa chọn

Nhóm tiêu chí	Bằng chứng kiểm chứng
Mã chung và metadata runtime	Phiên bản mã chung; đổi cấu hình A khi backend vẫn chạy; A đổi hành vi, B giữ nguyên.
Một instance, nhiều tenant	Yêu cầu A, B chồng lấn thời gian và cùng instance_id.
Database chung, cô lập logic	Cùng bảng có bản ghi A, B; thao tác chéo bị từ chối; context được dọn sau lỗi.

2.6.2. Giới hạn của phương án dùng chung

Một backend là điểm giới hạn năng lực và có thể là điểm gián đoạn chung. Database dùng chung làm việc khôi phục riêng một tenant và thay đổi lược đồ phức tạp hơn. Cấu hình sai hoặc truy vấn thiếu giới hạn có phạm vi ảnh hưởng rộng. Thiết kế cần nêu các giới hạn đó để diễn giải kết quả thử nghiệm trong đúng môi trường, tránh suy ra tính sẵn sàng cao từ demo chức năng.

2.6.3. Định hướng mở rộng ngang và phân bố dữ liệu

Hướng phát triển là đặt nhiều backend sau cân bằng tải, thống nhất trạng thái phiên và phiên bản metadata, rồi đánh giá mở rộng dữ liệu theo nút thắt thực tế. Database phân tán, replica hoặc phân vùng được xem xét theo yêu cầu tải và nhất quán [6]. Để xác nhận khả năng mở rộng phải có triển khai nhiều instance và phép đo tương ứng; phần thiết kế hiện tại không mặc nhiên đạt được năng lực đó.

2.7. Tổng kết chương

Chương 2 so sánh các mô hình Silo, Pool, Bridge và Hybrid, sau đó phân tích các cơ chế chia sẻ, cô lập và điều kiện đánh đổi. Phương án chung mã nguồn, backend và database được chọn từ phạm vi ứng dụng trong thuyết minh. Chương 3 chuyển lựa chọn này thành cấu trúc thành phần, dữ liệu, quyền, metadata và các luồng nghiệp vụ cụ thể.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ ỨNG DỤNG QUẢN LÝ CÔNG VIỆC ĐA THUÊ BAO

Chương này trình bày phương án phân tích và thiết kế ứng dụng Kanban trong môi trường đại học. Các yêu cầu trong thuyết minh được chuyển thành thành phần, dữ liệu, quyền và luồng xử lý cụ thể trên kiến trúc dùng chung có nhận biết tenant. Phạm vi chương là thiết kế và chuẩn bị kiểm chứng.

3.1. Phân tích bài toán và yêu cầu hệ thống

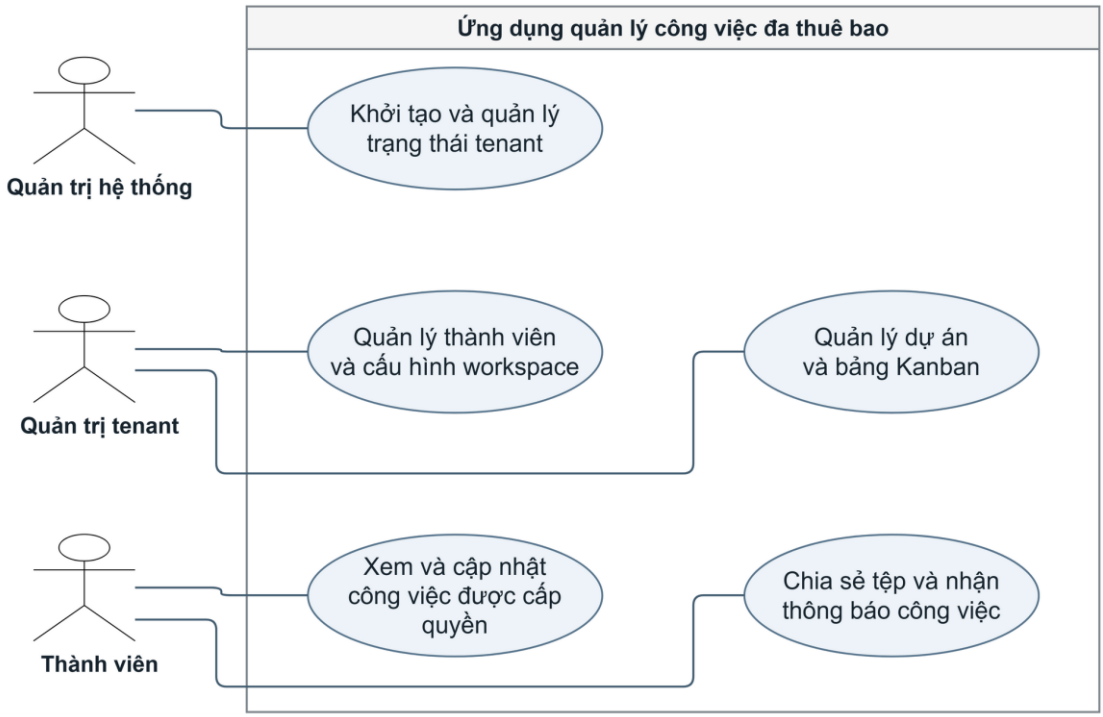
3.1.1. Bối cảnh sử dụng, các tác nhân và phạm vi ứng dụng

Ứng dụng hỗ trợ giảng viên và sinh viên quản lý công việc học tập, đồ án hoặc nghiên cứu theo Kanban. Một workspace đại diện tenant, bên trong có nhiều dự án, bảng, cột và thẻ công việc. Ba nhóm tác nhân là quản trị hệ thống, quản trị tenant và thành viên. Quản trị hệ thống quản lý vòng đời tenant; quyền nghiệp vụ trong workspace được xác định qua membership và phạm vi tài nguyên.

Phạm vi thiết kế gồm quản lý workspace, thành viên, dự án, công việc, tệp và thông báo cơ bản. Cập nhật trạng thái dùng giao dịch và kiểm soát phiên bản; không xây dựng đồng bộ thời gian thực phức tạp. Khởi tạo tenant tạo dữ liệu và cấu hình logic trong database chung, không cấp database hoặc backend riêng cho từng workspace.

3.1.2. Yêu cầu chức năng và sơ đồ ca sử dụng

Hình 3.1 và Bảng 3.1 xác định chức năng theo tác nhân. Quản trị tenant mời thành viên, quản lý quyền và cấu hình; thành viên xem hoặc cập nhật công việc theo phạm vi được giao. Thao tác xóa board dành cho quản trị tenant theo quy tắc thuyết minh. Các ca sử dụng được phân rã thành API và điều kiện kiểm tra ở backend, thay vì chỉ kiểm soát khả năng hiển thị trên giao diện.



Hình 3.1 Nhóm ca sử dụng theo vai trò của ứng dụng

Các đường nối biểu diễn tác nhân tham gia nhóm chức năng, không phải luồng thực thi. Quyền trên tài nguyên cụ thể được kiểm tra thêm theo membership và phạm vi công việc.

Bảng 3.1 Các nhóm yêu cầu chức năng

Mã	Chức năng	Phạm vi
F1	Quản lý tenant	Khởi tạo, tạm ngưng, cấu hình mặc định.
F2	Quản lý membership	Mời, gán vai trò, thu hồi quyền theo workspace.
F3	Quản lý dự án và board	Tạo, xem, cập nhật; xóa theo quyền quản trị.
F4	Quản lý công việc	Giao việc, cập nhật trạng thái và thời hạn.
F5	Tùy biến workspace	Tên, màu và quy tắc WIP qua metadata.
F6	Tài nguyên, thông báo	Tệp gắn công việc, nhắc thời hạn và thông báo cơ bản.

3.1.3. Yêu cầu phi chức năng

Các yêu cầu phi chức năng ưu tiên tính đúng đắn của cô lập, nhất quán trạng thái và khả năng kiểm chứng trên môi trường giới hạn. Mọi request phải gắn được

tenant đã xác minh; lỗi không để lại context dùng nhầm. Metadata được cập nhật khi backend tiếp tục chạy. Hiệu năng được đo theo workload và cấu hình thực tế, không đặt trước một kết quả độ trễ hay tỷ lệ tiết kiệm chưa có số đo.

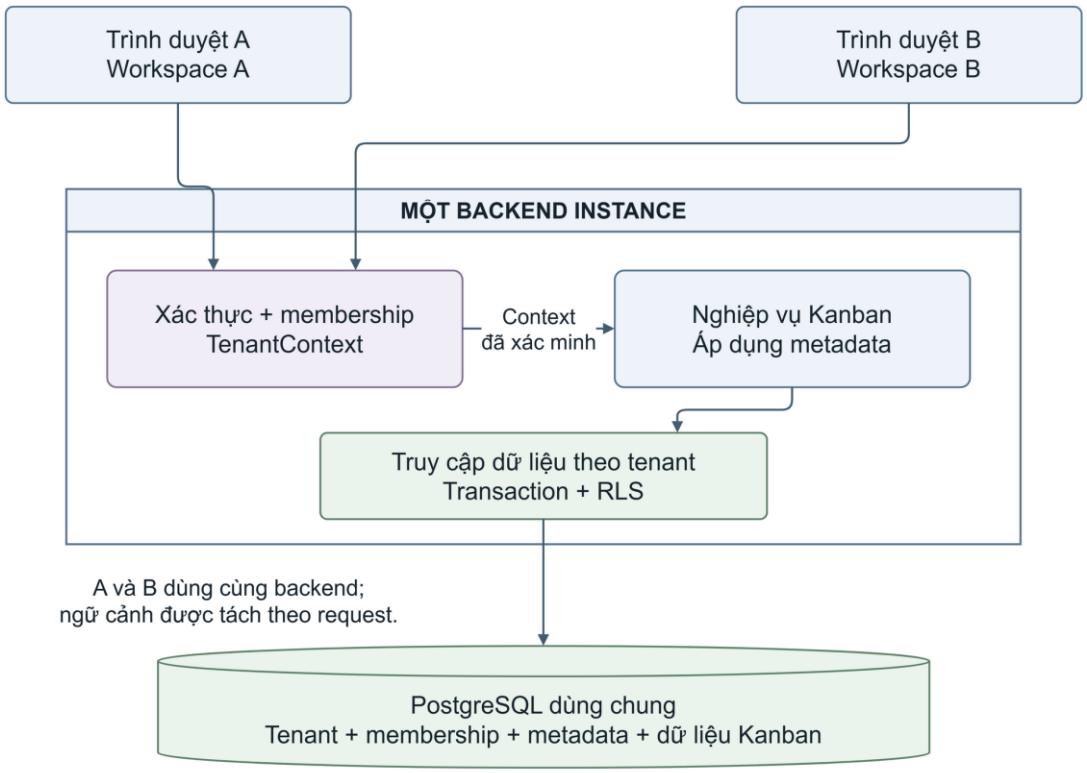
3.2. Thiết kế kiến trúc tổng thể

3.2.1. Kiến trúc có khả năng cấu hình và phục vụ đa thuê bao hiệu quả (Configurable, Multi-Tenant-Efficient)

Thiết kế lựa chọn kiến trúc Configurable, Multi-Tenant-Efficient đã phân tích ở Chương 2: chung mã, một backend phục vụ nhiều tenant và database dùng chung. Các khác biệt được hỗ trợ nằm trong metadata. Cách tổ chức này phù hợp nghiệp vụ Kanban tương đối đồng nhất và giúp kiểm chứng trực tiếp việc giữ riêng dữ liệu, cấu hình trong môi trường xử lý chung.

3.2.2. Các thành phần hệ thống và lựa chọn công nghệ

Hình 3.2 tổ chức backend thành các mô-đun cùng một tiến trình: xác thực và tenant, cấu hình, Kanban, dữ liệu và thông báo. Phương án công nghệ đề xuất là ứng dụng Web JavaScript, backend Java/Spring Boot, PostgreSQL và Web Server/reverse proxy phục vụ truy cập HTTPS. PostgreSQL phù hợp mô hình quan hệ và cho phép bổ sung chính sách RLS; các phiên bản cụ thể được chốt trong hồ sơ triển khai.



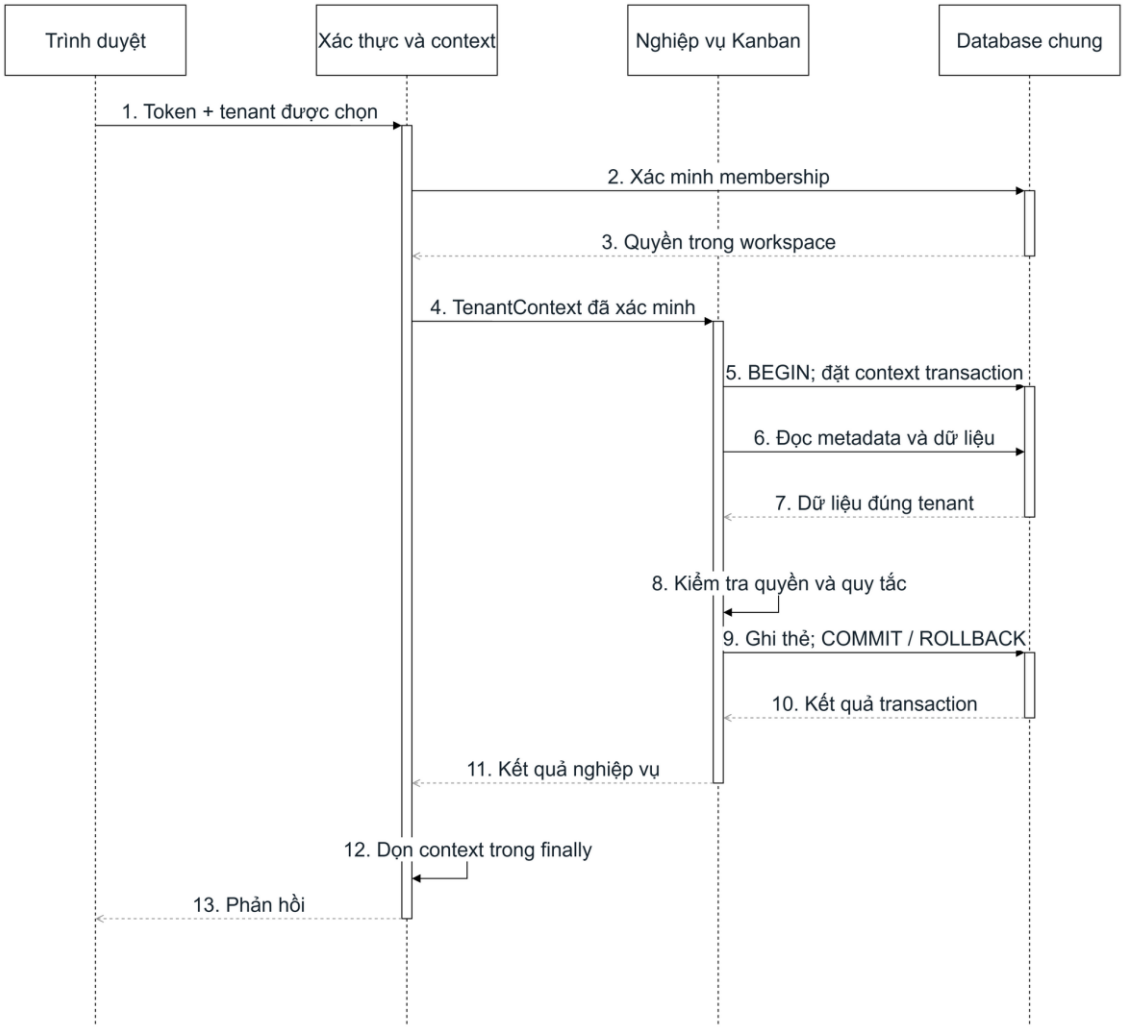
Hình 3.2 Kiến trúc thành phần của ứng dụng được đề xuất

Các thành phần bên trong khung backend là mô-đun cùng một instance. Cả hai trình duyệt đều qua xác thực và kiểm tra membership trước nghiệp vụ; database lưu chung danh mục, metadata và dữ liệu Kanban.

Thiết kế tối thiểu không yêu cầu Redis Cluster, Kubernetes hoặc hệ thống cờ tính năng bên ngoài. Metadata được lưu trong database và đọc theo tenant; cache có thể bổ sung khi phép đo cho thấy cần thiết. Cách chọn này giữ rõ cơ chế cốt lõi và giảm thành phần vận hành trên máy chủ thử nghiệm có tài nguyên giới hạn.

3.2.3. Mã nguồn chung, instance dùng chung và luồng xử lý yêu cầu

Hình 3.3 minh họa tạo thẻ trên backend chung. Bộ lọc xác thực user và đối chiếu membership với tenant được chọn trước khi thiết lập context. Nghiệp vụ mở transaction, đặt ngữ cảnh dữ liệu, đọc cấu hình và kiểm tra quyền, sau đó ghi thẻ với tenant_id do backend xác định. Giao dịch kết thúc bằng commit hoặc rollback; context luôn được dọn trong finally.



Hình 3.3 Luồng tạo thẻ có xác lập và giải phóng ngữ cảnh tenant

Sơ đồ biểu diễn nhánh hợp lệ; lỗi xác thực kết thúc trước nghiệp vụ. Backend gửi lệnh kết thúc transaction, nhận kết quả rồi dọn context trong finally. Việc dọn vẫn phải diễn ra khi có ngoại lệ.

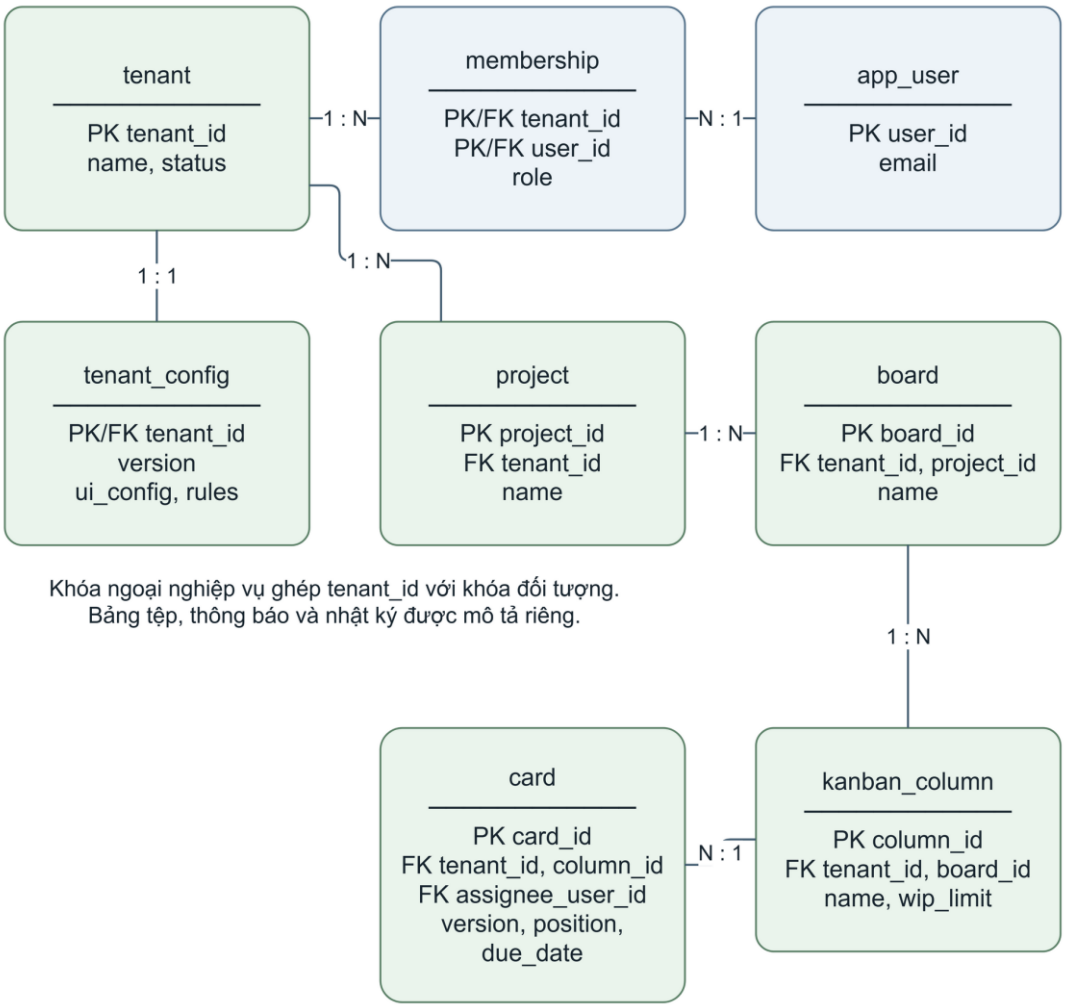
Yêu cầu A và B sử dụng cùng luồng mã nhưng có context, metadata và bản ghi tương ứng. Backend không nhận tenant_id trong body làm nguồn quyền. Những nhánh thiếu tenant, không có membership hoặc tenant ngừng hoạt động được từ chối trước thao tác nghiệp vụ. Log dùng request_id và instance_id để nối luồng xử lý với bằng chứng triển khai.

3.3. Thiết kế cơ sở dữ liệu

3.3.1. Mô hình dữ liệu nghiệp vụ và quan hệ giữa các thực thể

Hình 3.4 mô tả lược đồ cốt lõi. tenant đại diện workspace; app_user lưu tài khoản; membership nối user với tenant và vai trò. Một tenant có nhiều project, project có nhiều board, board có nhiều kanban_column và mỗi cột có nhiều card.

Thẻ có người được giao, thời hạn, vị trí và version phục vụ cập nhật có kiểm soát.



Hình 3.4 Lược đồ logic các thực thể dữ liệu cốt lõi

Các nhãn 1:N, N:1 đọc theo vị trí từ trái sang phải hoặc từ trên xuống dưới. Các khóa ngoại nghiệp vụ mang tenant_id; người được giao việc phải là thành viên hợp lệ của cùng workspace. Lược đồ trình bày nhóm thực thể cốt lõi.

Các bảng bổ trợ gồm attachment, notification và activity_log, đều mang tenant_id khi chứa dữ liệu tổ chức. Bảng 3.2 tóm tắt khóa và trách nhiệm của các thực thể. Tên bảng là đề xuất thiết kế và phải thống nhất với migration khi hiện thực. Không tạo một bộ bảng vật lý mới khi thêm workspace.

Bảng 3.2 Vai trò và ràng buộc của các nhóm dữ liệu

Nhóm bảng	Vai trò	Ràng buộc chính
tenant	Workspace và trạng thái	tenant_id duy nhất.
app_user, membership	Tài khoản và quan hệ thành viên	Cặp tenant_id, user_id không trùng.
tenant_config	Metadata và phiên bản cấu hình	Một bộ cấu hình theo tenant; giá trị hợp lệ.
project, board, kanban_column	Cấu trúc quản lý công việc	Khóa ngoại cha–con có cùng tenant_id.
card	Thẻ, người được giao, version	Cột và membership hợp lệ trong tenant.
attachment	Tài nguyên của công việc	Khóa tệp gắn tenant và quyền trên card.
notification, activity_log	Thông báo và lịch sử thao tác	Người nhận, đối tượng thuộc đúng tenant.

3.3.2. Phân tách dữ liệu và kiểm soát truy cập theo Tenant ID

Các bảng nghiệp vụ có tenant_id không rỗng; quan hệ cha–con tham chiếu bằng cặp tenant_id và khóa đối tượng. Ví dụ, card chỉ được trỏ đến kanban_column có cùng tenant_id. Dùng khóa duy nhất theo cặp và khóa ngoại tương ứng giúp database từ chối liên kết chéo tổ chức, bổ sung cho kiểm tra nghiệp vụ tại backend [9].

Truy cập theo ID luôn kết hợp tenant hiện hành và quyền trên tài nguyên. RLS được áp dụng cho bảng chứa dữ liệu tenant bằng vai trò ứng dụng không có quyền vượt chính sách. Mỗi transaction đặt context sau khi xác minh membership; tình huống thiếu hoặc rỗng context được xử lý theo nguyên tắc từ chối. Metadata cũng nằm trong phạm vi kiểm soát dữ liệu này.

3.3.3. Lưu trữ cấu hình và metadata theo tenant

tenant_config dùng tenant_id làm khóa và có version, ui_config cùng rules. ui_config lưu tên hiển thị, màu nhận diện hoặc lựa chọn trình bày; rules lưu các tính năng và quy tắc như wip_enabled, default_wip_limit. Schema của metadata do ứng dụng quy định, được kiểm tra kiểu và miền giá trị trước khi lưu. Cấu hình không cho phép chứa lệnh thực thi tùy ý.

Trong phương án tối thiểu, backend đọc cấu hình từ database theo ngữ cảnh khi xử lý nghiệp vụ. API giao diện trả metadata và version của workspace hiện hành; client nạp lại khi cập nhật hoặc chuyển workspace. Nếu bổ sung cache, khóa phải mang tenant_id và cơ chế invalidation phải được kiểm chứng cùng phép thử runtime.

3.4. Thiết kế các cơ chế đa thuê bao

3.4.1. Khởi tạo, nhận diện và quản lý ngữ cảnh tenant

Khởi tạo workspace được tổ chức trong một transaction gồm tạo tenant, membership quản trị, metadata mặc định và dữ liệu khởi đầu. Có lỗi thì rollback, tránh tenant thiếu cấu hình hoặc không có người quản lý. Trong phạm vi demo, quản trị hệ thống thực hiện cấp workspace; tự phục vụ có thể gọi lại cùng quy trình khi được bổ sung.

Request chọn workspace qua đường dẫn, subdomain hoặc header theo phương án triển khai đã chốt. Backend xác thực user, kiểm tra trạng thái tenant và membership rồi thiết lập TenantContext. Tra cứu membership thuộc đường xác thực giới hạn theo user đã xác thực và tenant được chọn; không mở quyền đọc chung các bảng nghiệp vụ. Context chỉ tồn tại trong phạm vi xử lý; chuyển workspace phải xác minh tenant đích và thay cả dữ liệu, cấu hình hiển thị.

3.4.2. Phân quyền và cô lập dữ liệu giữa các tenant

Phân quyền kết hợp vai trò trong workspace với quyền trên dự án, board hoặc thẻ. Quản trị tenant quản lý thành viên, cấu hình và tài nguyên của tổ chức mình; thành viên chỉ thao tác trong phạm vi được cấp. Quyền quản trị hệ thống không tự trở thành quyền đọc mọi thẻ qua API thông thường. Các hoạt động hỗ trợ đặc quyền, nếu có, phải được thiết kế và ghi nhận riêng.

Bảng 3.3 phân định quyền chức năng; kiểm tra tại service và giới hạn dữ liệu tại database bổ trợ nhau. Các tình huống user A lấy ID thẻ B, giả tenant trong header, ghi tenant_id khác trong body hoặc tạo liên kết cha–con chéo tenant đều phải có kết quả bị từ chối và không gây thay đổi dữ liệu.

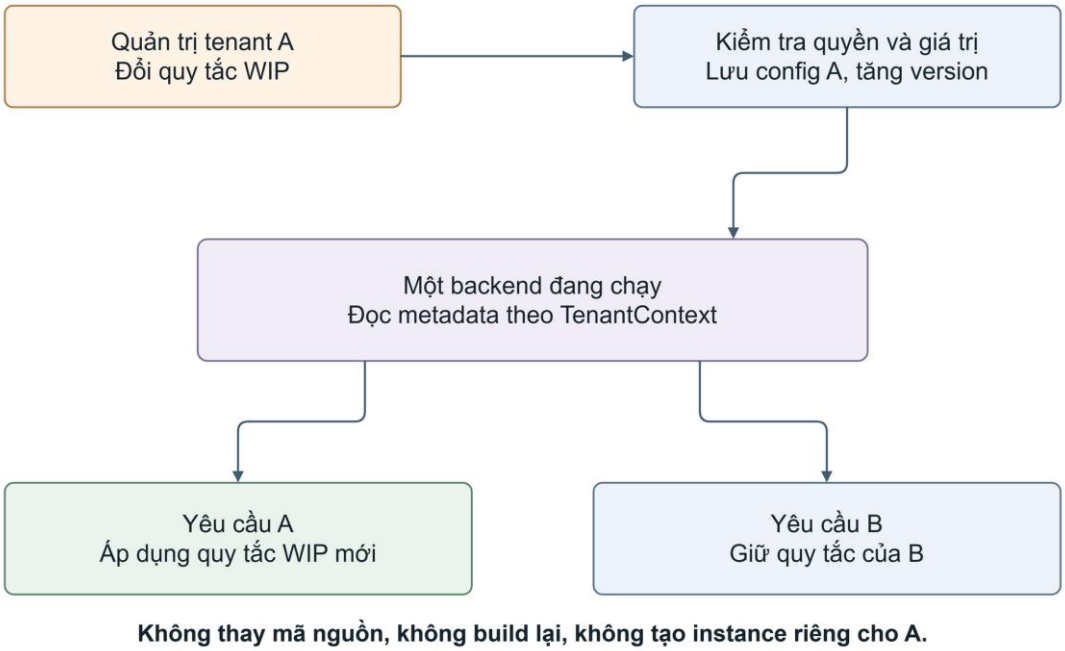
Bảng 3.3 Ma trận quyền chức năng trong phạm vi đề xuất

Thao tác	Quản trị hệ thống	Quản trị tenant	Thành viên
Khởi tạo, ngừng tenant	Có	Không	Không
Quản lý thành viên, metadata	Không qua API nghiệp vụ	Trong tenant mình	Không

Thao tác	Quản trị hệ thống	Quản trị tenant	Thành viên
Tạo, xóa board	Không qua API nghiệp vụ	Trong tenant mình	Không
Xem board, công việc	Không mặc định	Trong tenant mình	Theo phạm vi được cấp
Cập nhật thẻ, tệp	Không mặc định	Theo quyền trong tenant	Theo phạm vi được cấp

3.4.3. Tùy biến giao diện và nghiệp vụ lúc chạy qua metadata

Hình 3.5 mô tả quản trị tenant A cập nhật metadata khi backend đang hoạt động. Backend kiểm tra quyền và giá trị, lưu cấu hình rồi tăng version. Request sau của A chọn cấu hình mới; B tiếp tục dùng cấu hình B. Không thay mã nguồn, tạo instance theo tenant hoặc khởi động lại để áp dụng một biến thể đã hỗ trợ.



Hình 3.5 Cập nhật và áp dụng metadata theo tenant khi ứng dụng đang chạy

Cấu hình A được cập nhật trên backend đang chạy, sau đó được chọn bởi context A. B giữ cấu hình riêng. Hình minh họa hành vi thiết kế; hiệu lực runtime được xác nhận bằng ca kiểm chứng tương ứng.

Ví dụ nghiệp vụ là bật giới hạn WIP cho A và giữ tắt ở B. Khi chuyển thẻ vào cột đầy của A, backend từ chối theo quy tắc; cùng loại thao tác tại B theo cấu hình B vẫn có thể hợp lệ. Đổi tên hoặc màu workspace minh họa tùy biến giao diện.

Phép thử gọi API trực tiếp bảo đảm hiệu lực quy tắc không phụ thuộc việc ẩn nút.

3.5. Thiết kế chức năng và giao diện ứng dụng

3.5.1. Quản lý không gian làm việc và thành viên

Giao diện workspace hiển thị rõ tổ chức đang được thao tác và vai trò của user. Quản trị viên mời thành viên qua danh tính được xác nhận, gán vai trò và thu hồi quyền; một membership không được trùng cặp tenant_id, user_id. Khi membership bị thu hồi hoặc tenant bị tạm ngưng, request tiếp theo phải được kiểm tra lại theo chính sách phiên đã thiết kế.

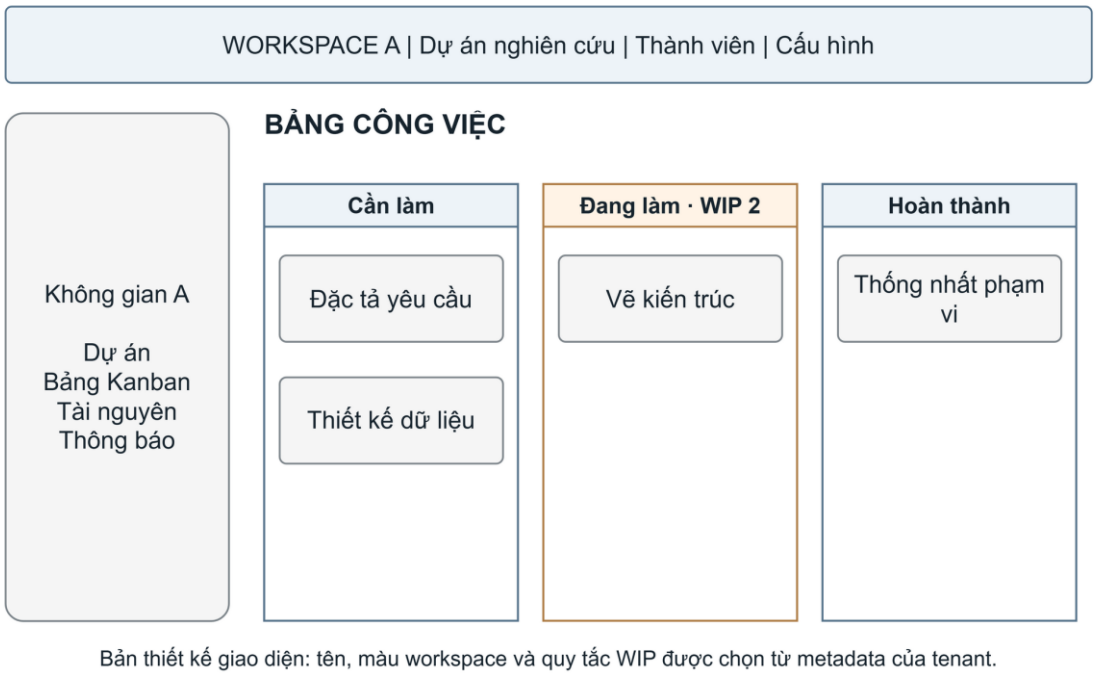
3.5.2. Quản lý dự án, bảng Kanban và trạng thái công việc

Dự án nhóm các board; board tổ chức cột trạng thái và card. Chuyển card kiểm tra cùng tenant, quyền trên board, cột đích và cấu hình WIP. Backend cập nhật vị trí, trạng thái và version trong cùng transaction. Cập nhật dựa trên version cũ bị từ chối có kiểm soát để tránh ghi đè thay đổi của người khác.

Khi nhiều request cùng đưa thẻ vào cột có giới hạn, kiểm tra đếm rồi ghi phải được phối hợp bằng cơ chế khóa phù hợp. Phương án đề xuất khóa hàng cột trước khi kiểm tra WIP, các thao tác liên quan tuân cùng giao thức và thứ tự khóa để hạn chế xung đột [10]. Phản hồi cuối cùng giúp client nạp lại trạng thái đã lưu; không đặt mục tiêu đồng bộ thời gian thực phức tạp.

3.5.3. Chia sẻ tài nguyên, thông báo và giao diện theo vai trò

Attachment lưu tenant_id, card_id và khóa tệp trong phạm vi workspace; backend kiểm tra quyền trước khi tải lên hoặc tải xuống. Tên đường dẫn không thay thế xác thực. Notification ghi nhận đúng tenant và người nhận; nhắc thời hạn được xử lý nền với ngữ cảnh tenant rõ ràng. Việc gửi thông báo bên ngoài chỉ thực hiện sau khi dữ liệu nghiệp vụ đã commit để tránh thông báo cho thao tác rollback.



Hình 3.6 Thiết kế giao diện quản lý công việc theo workspace

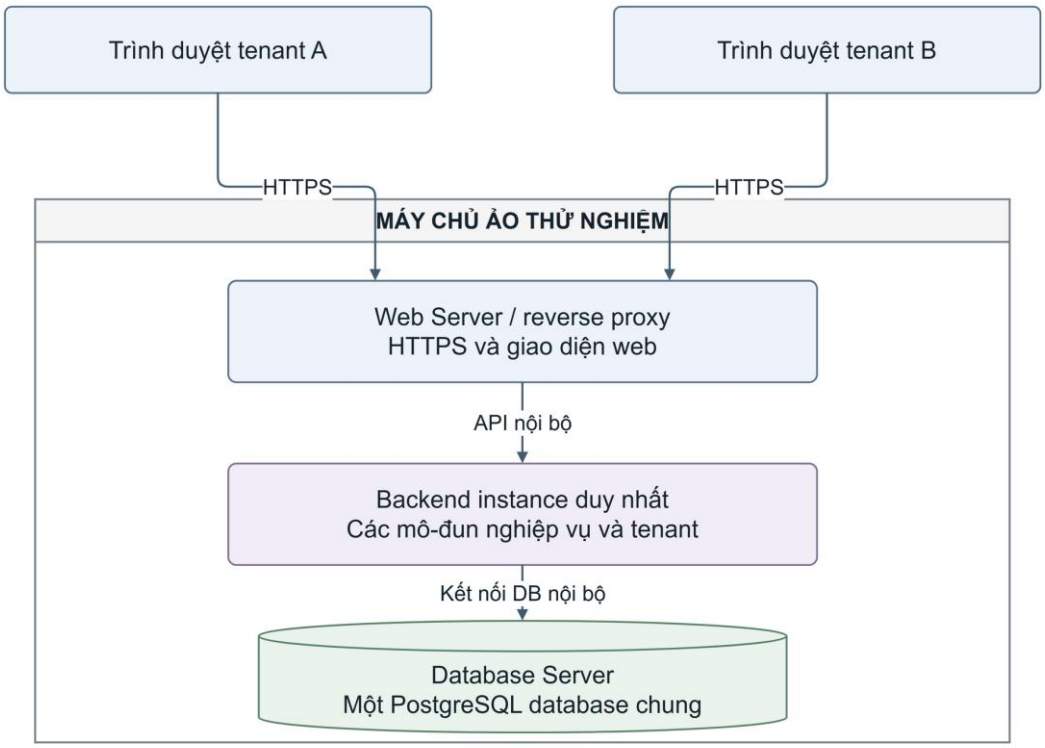
Wireframe thể hiện workspace hiện hành, dự án và các cột Kanban. Tên, màu và quy tắc WIP được chọn từ metadata; quyền thực hiện thao tác vẫn do backend quyết định.

Hình 3.6 là wireframe đề xuất cho bảng Kanban: thanh đầu xác định workspace và dự án, khu vực chính hiển thị các cột và thẻ, chức năng quản trị được trình bày theo vai trò. Màu, tên workspace và quy tắc WIP phản ánh metadata của tenant. Đây là bản thiết kế giao diện, không phải ảnh chứng minh ứng dụng đã hoạt động.

3.6. Thiết kế triển khai và kiểm chứng

3.6.1. Mô hình triển khai ứng dụng và cơ sở dữ liệu dùng chung

Hình 3.7 đề xuất triển khai một backend nghiệp vụ và PostgreSQL trên máy chủ ảo thử nghiệm; Web Server/reverse proxy cung cấp HTTPS và chuyển API vào backend. Database chỉ nhận kết nối từ môi trường ứng dụng. Tệp và dữ liệu cần có vị trí lưu bền vững và quy trình sao lưu phù hợp. Cấu hình CPU, RAM, phần mềm và phiên bản mã được ghi khi thiết lập thực tế.



Hình 3.7 Mô hình triển khai ứng dụng phục vụ nhiều tenant trên máy chủ thử nghiệm

Web Server và Database Server là các vai trò khác với backend nghiệp vụ. A và B truy cập cùng backend; không tạo container ứng dụng theo tenant. Luồng database là kết nối nội bộ trong môi trường máy chủ thử nghiệm.

Theo thuyết minh, quy mô dự kiến là 3–5 tenant, mỗi tenant khoảng 10–20 tài khoản hoạt động đồng thời. Đây là kế hoạch tải, không phải kết quả chịu tải đã đạt. Môi trường kiểm chứng phải giữ cùng instance backend và database khi thêm tenant, đồng thời thu thập log có tenant_id và instance_id để đối chiếu.

3.6.2. Đối chiếu yêu cầu thiết kế với tiêu chí kiểm chứng

Bảng 3.4 liên kết yêu cầu với thành phần và ca kiểm chứng. Trình tự demo dự kiến gồm khởi tạo A, B; tạo dữ liệu trong cùng bảng; gửi request chồng lấn vào một backend; đổi metadata A khi ứng dụng vẫn chạy; thử truy cập chéo tenant và tái sử dụng context sau lỗi. Mỗi ca lưu đầu vào, phản hồi và trạng thái dữ liệu trước, sau.

Kết quả kỳ vọng là điều kiện để đánh giá, chưa phải kết quả thực nghiệm. Việc xác nhận kiến trúc cần đủ bằng chứng về mã chung và metadata runtime, chia sẻ backend, database chung có cô lập logic. Những ca chưa thực hiện hoặc không đủ bằng chứng được ghi là chưa xác nhận; các phép đo hiệu năng và kết luận vận hành sẽ được trình bày khi có số liệu chạy thử.

Bảng 3.4 Ma trận thiết kế và ca kiểm chứng dự kiến

Mã	Thao tác kiểm chứng	Kết quả kỳ vọng
K1	Tạo A, B trên cùng triển khai	Không tạo backend hoặc DB theo tenant.
K2	Gửi request A, B chồng lấn	Cùng instance xử lý; dữ liệu không lẫn.
K3	Đổi tên/màu và quy tắc WIP của A	Có hiệu lực lúc chạy; B giữ cấu hình.
K4	Đọc, sửa, xóa ID đối tượng B bằng user A	Không vượt quyền; dữ liệu B không đổi.
K5	Thiếu hoặc giả tenant; body mang tenant khác	Từ chối hoặc giới hạn theo context tin cậy.
K6	Xen kẽ A, B sau lỗi và rollback	Không còn context A khi phục vụ B.
K7	Tạo liên kết dữ liệu chéo tenant	Ràng buộc hoặc nghiệp vụ từ chối.
K8	Hai user cập nhật cùng thẻ/cột giới hạn WIP	Không ghi đè mất cập nhật hoặc vượt giới hạn do cạnh tranh.

3.7. Tổng kết chương

Chương 3 cụ thể hóa kiến trúc được lựa chọn thành yêu cầu, mô-đun backend, dữ liệu quan hệ, ngữ cảnh tenant, phân quyền và metadata. Nghiệp vụ Kanban cung cấp các tình huống để quan sát cơ chế chia sẻ và cô lập. Mô hình triển khai và ma trận kiểm chứng tạo căn cứ cho hiện thực và thực nghiệm tiếp theo, đồng thời giữ rõ giới hạn giữa phương án thiết kế và kết quả đã quan sát.

TÀI LIỆU THAM KHẢO

[1] Mell, P. và Grance, T. (2011). The NIST Definition of Cloud Computing. NIST SP 800-145. https://doi.org/10.6028/NIST.SP.800-145.

[2] Chong, F. và Carraro, G. (2006). Architecture Strategies for Catching the Long Tail. Microsoft, April 2006. https://msdn.microsoft.com/en-us/library/aa479069.aspx.

[3] Kabbedijk, J., Bezemer, C.-P., Jansen, S. và Zaidman, A. (2015). Defining Multi-Tenancy: A Systematic Mapping Study on the Academic and the Industrial Perspective. Journal of Systems and Software, 100, 93–103. https://doi.org/10.1016/j.jss.2014.10.034.

[4] Bezemer, C.-P. và Zaidman, A. (2010). Multi-Tenant SaaS Applications: Maintenance Dream or Nightmare? Proceedings of IWPSE-EVOL. Bản tác giả: https://azaidman.github.io/publications/bezemerIWPSE2010.pdf.

[5] Amazon Web Services. SaaS Tenant Isolation Strategies, Core isolation concepts. https://docs.aws.amazon.com/whitepapers/latest/saas-tenant-isolation-strategies/core-isolation-concepts.html.

[6] Microsoft. Architectural approaches for storage and data in multitenant solutions. Azure Architecture Center. https://learn.microsoft.com/en-us/azure/architecture/guide/multitenant/approaches/storage-data.

[7] PostgreSQL Global Development Group. PostgreSQL 17 Documentation, Row Security Policies. https://www.postgresql.org/docs/17/ddl-rowsecurity.html.

[8] PostgreSQL Global Development Group. PostgreSQL 17 Documentation, SET. https://www.postgresql.org/docs/17/sql-set.html.

[9] PostgreSQL Global Development Group. PostgreSQL 17 Documentation, Constraints. https://www.postgresql.org/docs/17/ddl-constraints.html.

[10] PostgreSQL Global Development Group. PostgreSQL 17 Documentation, Explicit Locking. https://www.postgresql.org/docs/17/explicit-locking.html.

Các tài liệu kỹ thuật trực tuyến [5]–[10] được đối chiếu ngày 07/11/2026. Mô hình và sơ đồ trong báo cáo do nhóm biên soạn lại để giải thích hoặc đề xuất thiết kế; các bảng kiểm chứng không chứa số đo thực nghiệm.